



Konzeption, Implementierung und Evaluation eines Virtual-Tabletop-Plugins für Obsidian.md

Fabian Donatus Wolfgang Urbanek

Matrikelnummer: 667074

Abschlussarbeit

zur Erlangung des akademischen Grades

Bachelor of Science (B.Sc.)

im Studiengang Bachelor Wirtschaftsinformatik

der [FOM Hochschule für Oekonomie & Management](#)

Fabian Donatus Wolfgang Urbanek

Hinter der Höh 20

34630 Gilserberg

vorgelegt bei

Prof. Dr. Claudius Stern

Gilserberg, 01.01.2026

Zusammenfassung

Virtual-Tabletops (VTTs) digitalisieren traditionelles Tabletop-Rollenspiel durch interaktive Karten, Token-Management und Fog of War. Die vorliegende Arbeit untersucht, wie VTT-Plugins in Obsidians Electron-basierter Umgebung performant implementiert werden können und welche technischen Herausforderungen dabei auftreten.

Im Rahmen der Arbeit wurde Atlas VTT als vollständiges Plugin für Obsidian konzipiert und implementiert (Kapitel 3). Die technischen Entscheidungen wurden durch systematische Evaluation begründet: Ein Benchmark-Vergleich von Canvas-2D-Frameworks (Konva.js: 23 FPS, Fabric.js: 9 FPS) und WebGL-basiertem PIXI.js (60 FPS) bei 8000 Objekten führte zur Wahl von PIXI.js v8 (??). Das Grid-System wurde durch TilingSprite-Optimierung von mehreren hundert Draw-Calls auf einen einzigen reduziert (??). Das Token-Management nutzt automatisches Sprite Batching und Texture Caching (??).

Die Performance-Evaluation erfolgte durch ein automatisiertes Benchmark-System mit 500 Iterationen über drei Szenarien (0, 20, 100 Token), wie in ?? dokumentiert. Das Plugin erreicht konstante 120 FPS mit einer Frame Time von 8,33 ms – deutlich unter dem 60-FPS-Zielwert von 16,67 ms (??). Die Langzeit-Analyse identifizierte jedoch ein Speicherleck von 1,8 MB pro Iteration, das bei manuellen Tests nicht aufgefallen wäre.

Die Ergebnisse zeigen, dass Electron keine prinzipielle Barriere für performante VTT-Plugins darstellt (??). Die Herausforderungen liegen primär in der Speicherverwaltung und Bundle-Optimierung, nicht in der Rendering-Performance (??). Die dokumentierten Architekturmuster sind auf andere grafikintensive Obsidian-Plugins übertragbar.

Vorwort

Das in dieser Arbeit gewählte generische Maskulinum bezieht sich zugleich auf die männliche, die weibliche und andere Geschlechteridentitäten. Zur besseren Lesbarkeit wird auf die gleichzeitige Verwendung männlicher und weiblicher Sprachformen verzichtet. Alle Geschlechteridentitäten werden ausdrücklich mitgemeint, soweit die Aussagen dies erfordern.

In der vorliegenden Arbeit wird der Einsatz von künstlicher Intelligenz (KI), beispielsweise des Sprachverarbeitungsmodells ChatGPT, als Instrument zur Unterstützung bei Textformulierungen genutzt. Die tranformerbasierten Sprachmodelle werden als interaktives Tool zur Überprüfung und Verbesserung von wissenschaftlichen Texten eingesetzt, indem Vorschläge zur Sprachverfeinerung und zur logischen Strukturierung von Argumenten geboten werden.

Es ist wichtig zu betonen, dass die Verwendung von KI in dieser Arbeit transparent und ethisch verantwortungsvoll erfolgt. Alle durch die KI generierten Inhalte werden sorgfältig überprüft, um sicherzustellen, dass sie den wissenschaftlichen Standards entsprechen und einen Beitrag zur Forschung leisten. Die finale Verantwortung für den Inhalt der Arbeit liegt beim Autor.

Inhaltsverzeichnis

Abbildungsverzeichnis

Tabellenverzeichnis

Abkürzungsverzeichnis

API	application programming interface
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DnD	Dungeons & Dragons
FPS	Frames Per Second
GC	Garbage Collection
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
MB	Megabyte
MVP	Minimum Viable Product
TTRPG	Tabletop Role-Playing Game
VTT	Virtual Tabletop
WebGL	Web Graphics Library

1 Einleitung

Dieses Kapitel führt in die Thematik der vorliegenden Arbeit ein. Zunächst wird in Abschnitt 1.1 die Relevanz von Virtual-Tabletop-Tools im Kontext moderner Tabletop-Rollenspiele dargelegt und die Motivation für die Integration eines solchen Werkzeugs in Obsidian erläutert. Abschnitt 1.2 beschreibt die technischen Herausforderungen, die sich bei der Entwicklung rechenintensiver Plugins in Electron-basierten Anwendungen ergeben. Darauf aufbauend formuliert Abschnitt 1.3 die zentrale Forschungsfrage sowie die daraus abgeleiteten Teilziele. Abschließend gibt Abschnitt 1.4 einen Überblick über die Struktur der Arbeit.

1.1 Hinführung zum Thema und Motivation

Ein Tabletop Role-Playing Game (TTRPG) ist ein interaktives Erzählerlebnis, bei dem Spieler die Rollen von Charakteren in einer gemeinsamen Welt übernehmen und zusammenarbeiten, um eine Geschichte über diese Charaktere zu erzählen [Daggerheart2025]. Typischerweise übernimmt eine Person die Rolle des Spielleiters (Game Master), der die Geschichte moderiert, Konsequenzen für Spielerentscheidungen festlegt und Nicht-Spieler-Charaktere verkörpert, während die anderen Spieler jeweils einen Charakter steuern. Würfel liefern dabei ein Element der Unvorhersehbarkeit für die Ergebnisse von Entscheidungen [Daggerheart2025]. Seit der Veröffentlichung von Dungeons & Dragons (DnD) im Jahr 1974 haben sich TTRPGs zu einem kulturellen Phänomen mit globaler Reichweite entwickelt: Für 2025 wird der weltweite Marktwert auf etwa 2,15 Milliarden US-Dollar geschätzt [BusinessResearchInsights2025].

Die zunehmende Digitalisierung und geografische Verteilung von Spielgruppen hat Virtual Tabletop (VTT)s zu einem verbreiteten Werkzeug moderner TTRPGs gemacht. Plattformen wie Roll20, Foundry VTT und Fantasy Grounds haben sich etabliert, wobei viele Gruppen auch nach der COVID-19-Pandemie VTTs für Spielsitzungen mit geografisch verteilten Teilnehmern nutzen [RPGDrop2024Market]. VTTs ermöglichen es, die klassischen Elemente des Tabletop-Rollenspiels – taktische Karten, Charakterbewegungen, Würfelwürfe und gemeinsames Storytelling – in einer digitalen Umgebung zu replizieren und dabei geografische Distanzen zu überbrücken.

Akademische Literatur zur technischen Implementierung und Architektur von VTT-Systemen ist kaum vorhanden. Eine bibliometrische Analyse von über 3.000 wissenschaftlichen Publikationen zu Tabletop-Rollenspielen zeigt, dass die For-

schung primär auf Psychologie, Soziologie und Bildungsanwendungen fokussiert ist [Martinolli2025TTRPGBibliometrie]. Technische Aspekte wie Software-Architektur, Rendering-Optimierung oder Plugin-Entwicklung werden in der akademischen Literatur nicht behandelt. Vereinzelte Arbeiten untersuchen VTTs als Beispiele für nutzerzentriertes Design [Szymczyk2022VTTVernacular], jedoch nicht deren technische Implementierung. Das Wissen zur VTT-Entwicklung existiert daher primär in Praktiker-Communities: Offizielle Dokumentationen wie die von Foundry VTT [FoundryVTT2024], technische Blogposts und Open-Source-Repositories bilden die maßgeblichen Quellen für Implementierungsdetails. Die vorliegende Arbeit stützt sich entsprechend auf diese webbasierten Quellen, ergänzt durch etablierte Literatur zu allgemeinen Software-Engineering-Prinzipien und Performance-Optimierung.

Obsidian.md hat sich in der TTRPG-Community als verbreitetes Werkzeug für Spiel-leiter etabliert, insbesondere für Worldbuilding und Session-Vorbereitung. Die Plattform wird bereits umfassend für die Organisation von Kampagnennotizen, Charakterbeschreibungen und Regelreferenzen genutzt. Das Plugin Fantasy Statblocks, das Statblocks für verschiedene Spielsysteme direkt in Obsidian rendert, verzeichnet über 250.000 Downloads [ValentineFantasyStatblocks2024]. Die Community hat ein umfangreiches Ökosystem spezialisierter Plugins entwickelt, darunter Initiative Tracker für Kampfbegegnungen [JavalentInitiativeTracker2024] und Dice Roller für Würfelsimulationen. Diese etablierte Nutzerbasis und die vorhandene Infrastruktur legen nahe, dass Obsidian eine geeignete Plattform für die Integration vollwertiger VTT-Funktionalität darstellen könnte.

Das im Rahmen dieser Arbeit zu entwickelnde VTT-Plugin zielt darauf ab, Virtual-Tabletop-Funktionalität in Obsidians etabliertes Markdown-Ökosystem zu integrieren. Unter Verwendung der Rendering-Engine PIXI.js v8 soll das Plugin interaktive Spielkarten, Token-Management und grundlegende Würfelfunktionalität direkt in der Obsidian-Umgebung ermöglichen. Die angestrebte Lösung würde es Spielleitern erlauben, ihre Kampagnennotizen, Weltenbeschreibungen und Regelreferenzen in derselben Anwendung zu verwalten, in der auch die taktischen Begegnungen durchgeführt werden. Diese Integration soll den Kontextwechsel zwischen verschiedenen Anwendungen reduzieren und einen kohärenten digitalen Arbeitsbereich für Tabletop-Rollenspiele schaffen. Ein zusätzlicher Vorteil dieses Ansatzes liegt in der Datenhoheit der Nutzer: Im Gegensatz zu proprietären VTT-Plattformen, bei denen Inhalte auf externen Servern gespeichert werden, verbleiben sämtliche erstellten Materialien als lokale Dateien im Dateisystem der Anwender. Dies entspricht der grundlegenden Designphilosophie von Obsidian, das auf einfachen Markdown-Dateien basiert und Nutzern die vollständige Kontrolle über ihre Daten gewährleistet [ObsidianAbout2024]. Die erstellten Kampagneninhalte bleiben damit dauerhaft zugänglich und unabhängig von der Verfügbarkeit externer Dienste.

1.2 Darstellung der Problemstellung

Die technische Umsetzung von VTT-Plugins in Electron-basierten Anwendungen wie Obsidian bringt spezifische Performance-Herausforderungen mit sich. Electron-Anwendungen basieren auf Chromium und erfordern sorgfältige Optimierung, um eine akzeptable Performance zu erreichen [ElectronPerformanceGuide2024]. Bei VTT-Plugins sind die Anforderungen technisch anspruchsvoll: Echtzeit-Rendering von komplexen Karten mit potenziell hunderten von Token, flüssige Animationen für Bewegungen und Effekte sowie die gleichzeitige Verwaltung von Spielerzuständen oder diverser anderer Kartenobjekte.

Die Speicher- und Central Processing Unit (CPU)-Beschränkungen von Electron könnten zu messbaren Performance-Einbußen führen, insbesondere bei längeren Spielsitzungen oder bei der Verwendung hochauflösender Kartenmaterialien. Frame-Drops unter 30 Frames Per Second (FPS) könnten die Interaktionsqualität beeinträchtigen, ebenso hohe Eingabelatenzen die Bedienbarkeit (die konkreten Schwellenwerte werden in Abschnitt 3.1 definiert). Zusätzlich entstehen Skalierungsprobleme bei der Verwaltung großer Datenmengen, etwa wenn die Anwendung lange Zeit verwendet wurde und hunderte von Karten, NPCs und Items verwaltet werden müssen.

Bestehende VTT-Lösungen für Obsidian beschränken sich auf grundlegende Visualisierungen ohne Echtzeit-Rendering-Anforderungen. Die Integration von modernen Rendering-Technologien wie Web Graphics Library (WebGL) über PIXI.js v8 bietet zwar das Potenzial für bessere Performance, erfordert jedoch eine sorgfältige Architektur und Optimierung, um die Limitierungen der Electron-Umgebung zu überwinden. Die Herausforderung besteht darin, ein Gleichgewicht zwischen funktionalem Umfang, visueller Qualität und akzeptabler Performance zu finden.

1.3 Zielsetzung und Forschungsfrage

Das Ziel dieser Arbeit ist die systematische Untersuchung der Performance-Charakteristiken von VTT-Plugins in Obsidian. Durch eine detaillierte Performance-Evaluation der implementierten Lösung und die Analyse der Rendering-Pipeline sollen Erkenntnisse für die Entwicklung performanter Plugin-Architekturen gewonnen werden.

Die zentrale Forschungsfrage lautet: *Wie können Virtual-Tabletop-Plugins in der Electron-basierten Umgebung von Obsidian so implementiert werden, dass sie trotz der technischen Limitierungen eine für Echtzeit-Interaktionen ausreichende Performance erreichen?*

Daraus ergeben sich folgende Teilziele:

- Evaluation verschiedener Rendering-Frameworks (Canvas-2D, WebGL) und fundierte Auswahl für die VTT-Implementierung
- Durchführung reproduzierbarer Performance-Benchmarks

- Quantitative Evaluation der Performance-Charakteristiken der implementierten Lösung
- Identifikation von Optimierungspotenzialen und Best Practices

Die gewonnenen Erkenntnisse sind primär für die Entwicklung des VTT-Plugins relevant, könnten jedoch auch für andere rechenintensive Obsidian-Plugins von Interesse sein. Angesichts der wachsenden Bedeutung von digitalen Werkzeugen für Tabletop-Rollenspiele und der in Abschnitt 1.1 dokumentierten Nutzung von Obsidian als TTRPG-Plattform adressiert diese Arbeit ein praxisrelevantes Problem mit direktem Nutzen für die bestehende Nutzergemeinschaft.

1.4 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in fünf Hauptkapitel:

In Kapitel 2 werden die theoretischen Grundlagen erarbeitet. Dies umfasst die konzeptuellen Grundlagen von Virtual-Tabletop-Tools und Plugin-Architekturen und die technischen Rahmenbedingungen von Obsidian und Electron.

Kapitel 3 beschreibt die Konzeption und Implementierung des VTT-Plugins. Hier werden die Anforderungsanalyse, das Systemdesign und verschiedene Lösungsansätze dokumentiert.

In ?? erfolgt die Evaluation der implementierten Lösung. Die durchgeführten Performance-Messungen werden ausgewertet und verschiedene Optimierungsstrategien verglichen.

?? fasst die wesentlichen Erkenntnisse zusammen, beantwortet die Forschungsfrage und gibt einen Ausblick auf zukünftige Entwicklungsmöglichkeiten.

2 Theoretische Grundlagen

Dieses Kapitel legt die theoretischen Grundlagen für die Entwicklung eines VTT-Plugins in Obsidian. Abschnitt 2.1 erläutert zunächst die konzeptuellen Grundlagen von Virtual-Tabletop-Tools und Plugin-Architekturen. Darauf aufbauend beschreibt Abschnitt 2.2 die technischen Rahmenbedingungen, die sich aus der Obsidian-Plattform, dem Electron-Framework und der gewählten Rendering-Engine PIXI.js v8 ergeben.

2.1 Konzeptuelle Grundlagen

Dieser Abschnitt führt in die grundlegenden Konzepte ein, die für das Verständnis eines VTT-Plugins erforderlich sind. Abschnitt 2.1.1 beschreibt die Kernkomponenten von Virtual-Tabletop-Tools und deren technische Anforderungen, während Abschnitt 2.1.2 die architektonischen Muster erläutert, die modernen Plugin-Systemen zugrunde liegen.

2.1.1 VTT-Tools

VTTs sind digitale Anwendungen, die die physische Spielumgebung von Pen-and-Paper-Rollenspielen in einem digitalen Raum simulieren [Roll20TokenFeatures2024]. Sie ersetzen die essenziellen Elemente des Tabletop-Rollenspiels – Karten, Miniaturen, Würfel und Charakterbögen – durch interaktive digitale Komponenten und ermöglichen räumlich getrennten Spielgruppen eine gemeinsame, synchronisierte Spielumgebung.

2.1.1.1 Kernkomponenten

Die zentralen visuellen Elemente eines VTT umfassen *Token*, *Statblocks* und *Fog of War*. Token sind die digitalen Entsprechungen klassischer Miniaturen und repräsentieren Spielercharaktere, Gegner oder Objekte auf der Spielkarte [Roll20TokenFeatures2024]. Abbildung 2.1 zeigt die Token-Implementierung in Atlas VTT mit Ressourcenbalken, Statuseffekten und Kontextmenü.

Statblocks enthalten strukturiert alle spielrelevanten Informationen einer Spielfigur – Attribute, Fähigkeiten und Aktionen – und ermöglichen in modernen VTTs interaktive Würfelwürfe direkt aus dem Datenblatt [DnDBeyondMonsters2024]. Abbildung 2.2 zeigt die Statblock-Implementierung in Atlas VTT, die Obsidians Markdown-Fähigkeiten nutzt.

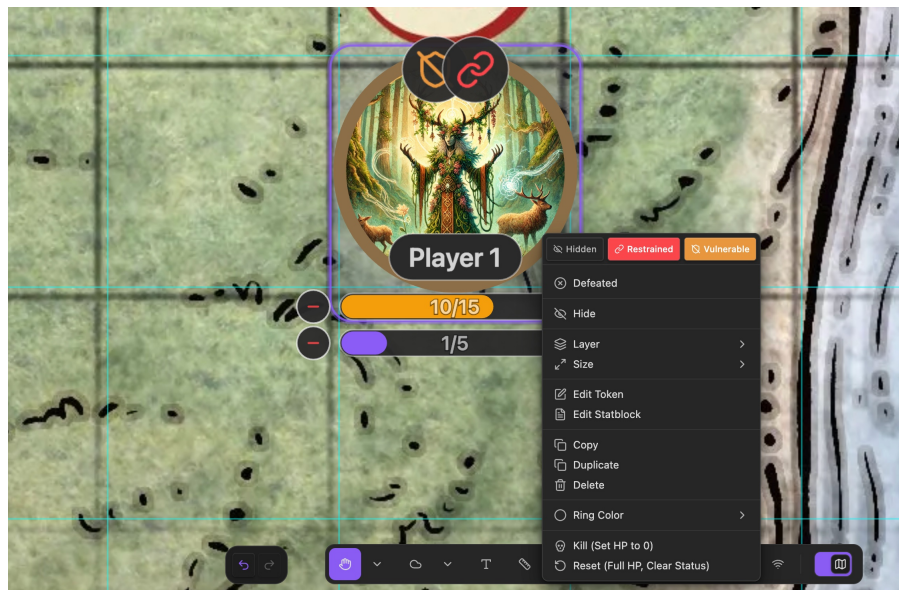


Abbildung 2.1: Token-Interaktion in Atlas VTT mit Ressourcenbalken, Statusseffekten und Kontextmenü.

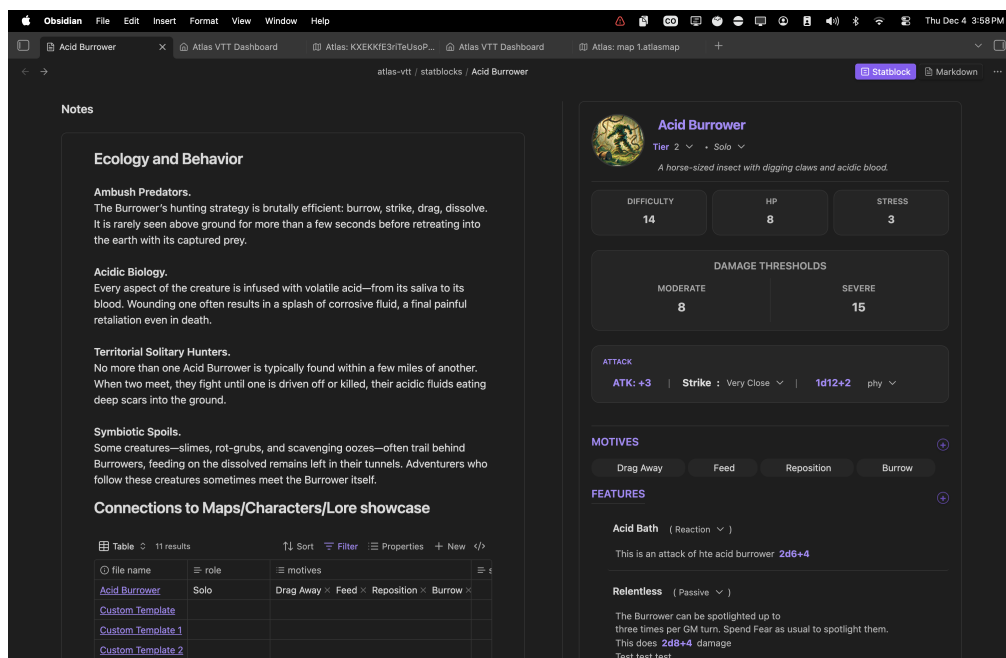


Abbildung 2.2: Statblock-Integration in Atlas VTT: Markdown-Quelldatei (links) und gerenderter Statblock (rechts).



Abbildung 2.3: Fog of War in Atlas VTT: Unerforschte Bereiche werden für Spieler verborgen.

Der Fog of War simuliert begrenzte Wahrnehmung, indem unerforschte oder aktuell nicht sichtbare Bereiche für Spieler verborgen werden [Roll20FogOfWar2024]. Abbildung 2.3 illustriert diese Komponente in Atlas VTT.

Neben den visuellen Komponenten benötigen VTTs eine zentrale Verwaltung für Spielressourcen wie Token-Grafiken, Kartenbilder und Statblocks. In Atlas VTT übernimmt diese Funktion der *Asset Manager*, dessen Implementierung Abbildung 2.4 zeigt.

2.1.1.2 Performance-Anforderungen

Für eine flüssige Spielerfahrung definiert Nielsen 100 Millisekunden als Schwellenwert, unterhalb dessen Systemreaktionen als augenblicklich wahrgenommen werden [Nielsen1993ResponseTimes]. Bei kontinuierlichen Interaktionen wie dem Ziehen von Token gelten niedrigere Schwellenwerte, wie in Abschnitt 3.1 detailliert dargelegt wird.

2.1.1.3 Marktübersicht

Die beiden Marktführer Roll20 und Foundry VTT verfolgen unterschiedliche Architekturansätze: Roll20 funktioniert vollständig browserbasiert, während Foundry VTT lokal gehostet wird und dadurch mehr Kontrolle und Anpassungsmöglichkeiten bietet [FoundryVTT2024]. Atlas VTT verfolgt als Obsidian-Plugin einen dritten Ansatz, bei dem die VTT-Funktionalität in eine bestehende Wissensmanagement-Anwendung integriert wird.

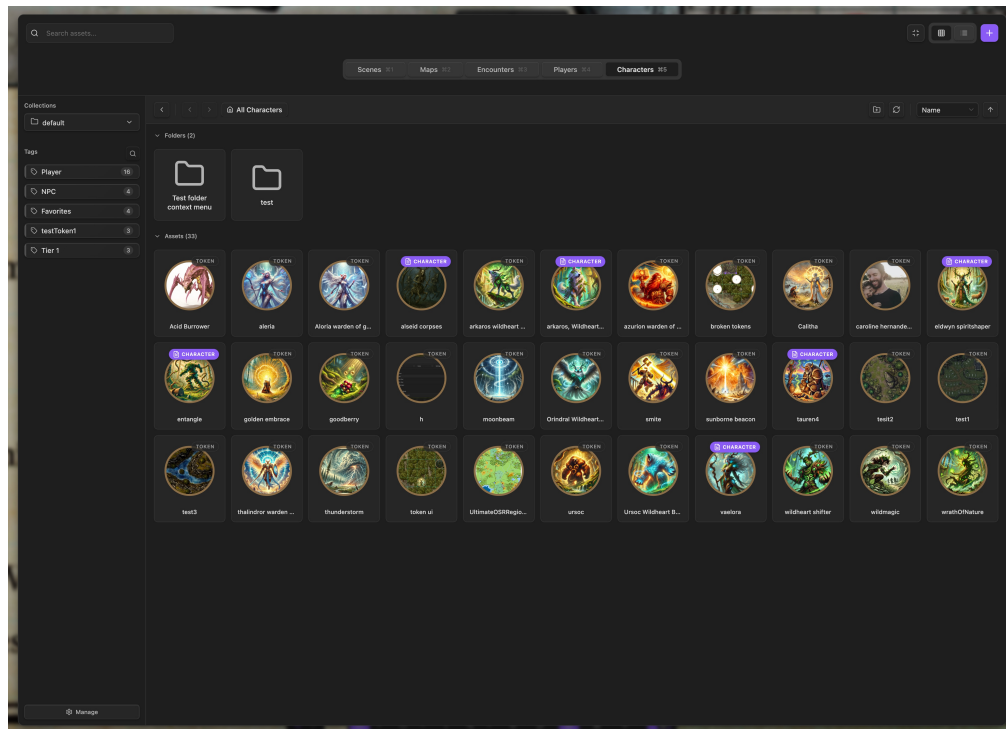


Abbildung 2.4: Asset Manager in Atlas VTT mit Collection-Sidebar, Tab-Navigation und Grid-Ansicht.

2.1.2 Plugin-Architekturen

Plugin-Architekturen ermöglichen die Erweiterbarkeit von Anwendungen ohne Modifikation des Kernsystems. Das Architekturmuster basiert auf der Trennung zwischen einem stabilen Kernsystem und dynamisch ladbaren Erweiterungsmodulen, die über wohldefinierte Schnittstellen kommunizieren. Das *Microkernel Pattern* strukturiert dabei die Anwendung in einen minimalen Kern mit essenziellen Funktionalitäten und Plugin-Module für spezifische Features [Richards2024MicrokernelArchitecture].

Moderne Plugin-Systeme nutzen häufig eine *Event-Driven Architecture*, bei der Komponenten asynchron über Events kommunizieren [Microsoft2024EventDriven]. Plugins registrieren Event-Handler für spezifische Ereignisse und können selbst Events auslösen – eine lose Kopplung, die Integration ohne direkte Abhängigkeiten ermöglicht. Das *Lifecycle Management* folgt einem definierten Zustandsmodell mit Phasen wie Loading, Initialization, Activation und Unloading, wobei jede Phase Hooks für Setup- und Cleanup-Operationen bietet [VSCodeExtensionLifecycle2024].

Für die Datenpersistierung nutzen Plugin-Systeme typischerweise isolierte Datenspeicher pro Plugin [VSCodeExtensionLifecycle2024]. Die Kommunikation zwischen Plugins erfolgt über Event-basierte Mechanismen statt direkter Aufrufe, um unerwünschte Abhängigkeiten zu vermeiden [RefactoringGuruObserver]. Performance-Überlegungen sind relevant, da jedes Plugin den Memory-Footprint erhöht – Strategien wie Lazy Loading optimieren die Ressourcennutzung [JavaDesignPatternsLazyLoading].

2.2 Technische Rahmenbedingungen

Die Entwicklung eines VTT-Plugins für Obsidian erfordert ein Verständnis der zugrundeliegenden technischen Plattformen. Abschnitt 2.2.1 beschreibt die Architektur von Obsidian und dessen Plugin-System. Abschnitt 2.2.2 erläutert das Electron-Framework, auf dem Obsidian basiert, sowie dessen Performance-Charakteristiken. Abschnitt 2.2.3 stellt PIXI.js v8 als Rendering-Engine vor und analysiert die für VTT-Anwendungen relevanten Features.

2.2.1 Obsidian als Markdown-Editor

Obsidian ist eine proprietäre Wissensdatenbank-Anwendung, die auf dem Konzept vernetzter Markdown-Dokumente basiert. Notizen werden als Plain-Text-Dateien innerhalb einer sogenannten *Vault*-Ordnerstruktur persistiert [ObsidianWiki2024]. Diese Architekturscheidung gewährleistet Datenportabilität und ermöglicht die Integration externer Werkzeuge für Versionskontrolle und Synchronisierung. Obsidian positioniert sich damit als persönliches Wissensmanagementsystem, dessen Kernfunktionen in bidirektionaler Verlinkung und Visualisierung von Wissensstrukturen liegen.

Das Plugin-System unterscheidet zwei Kategorien: *Core Plugins*, die vom Obsidian-Team entwickelt werden, sowie *Community Plugins*, die als Open-Source-Erweiterungen über GitHub distribuiert werden [ObsidianWiki2024]. Die Plugin-application programming interface (API) folgt einer Event-basierten Architektur, die lose Kopplung zwischen Plugins und Kernsystem gewährleistet [ObsidianEvents2024]. Plugins registrieren Handler für spezifische Systemereignisse und werden asynchron benachrichtigt – ein Muster, das dem in Abschnitt 2.1.2 beschriebenen Observer-Pattern entspricht.

Für ein VTT-Plugin sind drei Aspekte der Obsidian-Architektur relevant: Das View-System ermöglicht das Rendern benutzerdefinierter Oberflächen in dedizierten Bereichen, die Datenpersistierung speichert Plugin-Konfigurationen, und die Vault-API gestattet Zugriff auf Markdown-Dateien [ObsidianVault2024]. Diese Kombination ermöglicht die Integration von Spielinhalten wie Statblocks oder Encounter-Definitionen aus regulären Notizen. Die konkreten Implementierungsdetails dieser Integration werden in Kapitel 3 erläutert.

2.2.2 Electron Framework

Als Open-Source-Framework für plattformübergreifende Desktop-Applikationen vereint Electron Webtechnologien (HyperText Markup Language (HTML), Cascading Style Sheets (CSS), JavaScript) mit nativen Betriebssystem-Capabilities. Die technologische Basis bildet die Kombination aus Chromiums Rendering Engine und der Node.js-Runtime [ElectronWiki2024]. Da Electron die technologische Grundlage von Obsidian darstellt, determiniert seine Architektur die Performance-Charakteristika eines darauf basierenden VTT-Plugins.

Architektonisch folgt Electron dem Multi-Process-Modell von Chromium, das zwischen dem *Main Process* und einem oder mehreren *Renderer Processes* differenziert [ElectronProcessModel2024]. Der Main Process verwaltet native Betriebssystem-Interaktionen, während Renderer Processes die Benutzeroberfläche darstellen. Diese Prozessisolation verbessert Stabilität und Sicherheit, erfordert jedoch explizite Kommunikationsmechanismen zwischen den Prozessen [ElectronIPC2024].

Für Performance-sensitive Anwendungen wie VTTs ist die Garbage Collection der V8 JavaScript Engine relevant: Major Garbage Collection (GC)-Zyklen können Latenzen verursachen, die sich als Frame-Drops manifestieren [V8MemoryOptimization2016]. Bei einer Zielframerate von 60 FPS steht pro Frame ein Zeitbudget von 16,6 Millisekunden zur Verfügung – Pausen durch Garbage Collection können dieses Budget überschreiten und die Flüssigkeit von Animationen beeinträchtigen.

2.2.3 PIXI.js v8 als Rendering-Engine

Für die Darstellung interaktiver 2D-Grafiken in Webanwendungen stehen verschiedene Rendering-Technologien zur Verfügung. Abschnitt 2.2.3.1 erläutert zunächst die grundle-

genden Browser-APIs für grafisches Rendering, bevor Abschnitt 2.2.3.2 die Architektur und Features von PIXI.js v8 als gewählte Rendering-Engine beschreibt.

2.2.3.1 Browser-basierte Rendering-APIs

Moderne Webbrowser stellen verschiedene APIs für grafisches Rendering bereit, die sich in ihrer Ausführungsarchitektur unterscheiden:

Die **HTML5 Canvas API** bietet eine Bitmap-Zeichenfläche für 2D-Rendering [WikiCanvas2024]. Zeichenoperationen werden auf der CPU ausgeführt, was bei objektreichen Szenarien zu Performance-Limitierungen führen kann.

WebGL (Web Graphics Library) ist eine JavaScript-API für Hardware-beschleunigtes 2D- und 3D-Rendering, die auf OpenGL ES basiert [WikiWebGL2024]. Im Gegensatz zur Canvas-API nutzt WebGL die Graphics Processing Unit (GPU) des Systems, wodurch mehr Objekte parallel verarbeitet werden können.

WebGPU stellt die nächste Generation der Browser-GPU-APIs dar und ist als Nachfolger von WebGL konzipiert [WikiWebGPU2024]. Die API bietet verbesserte Kompatibilität mit modernen GPUs und befindet sich seit 2024 im W3C Candidate Recommendation Status.

2.2.3.2 PIXI.js als Rendering-Engine

PIXI.js ist eine Open-Source-2D-Rendering-Engine, die Low-Level-WebGL-APIs durch eine objektorientierte Schnittstelle abstrahiert. Version 8 (März 2024) führt WebGPU als zusätzlichen Renderer ein und bietet Performance-Optimierungen gegenüber früheren Versionen [PixiJS2024Launch]. Für VTT-Plugins bildet diese Engine das Fundament für Map-Rendering, Token-Visualisierung und interaktive Spielelemente.

Konzeptionell organisiert PIXI.js Grafikobjekte in einem *Scene-Graph* – einer hierarchischen Baumstruktur, in der Parent-Objekte ihre Transformationen (Position, Rotation, Skalierung) automatisch an Child-Objekte weitergeben [PixiJSSceneGraph2024]. Diese Komposition ermöglicht flexible Objektgruppierung: Ein Token kann beispielsweise aus mehreren visuellen Elementen bestehen, die sich gemeinsam bewegen.

Die zentrale Performance-Optimierung der Engine ist *Batch-Rendering*: Grafikobjekte mit identischen Eigenschaften werden in einem einzigen GPU-Aufruf verarbeitet, anstatt separate Aufrufe pro Objekt durchzuführen. Zusätzlich implementiert Version 8 ein Dirty-Flag-System, das nur modifizierte Objekte aktualisiert – statische Szenen verursachen dadurch minimalen CPU-Overhead [PixiJSV8Performance2024]. Diese Eigenschaften sind für VTT-Anwendungen relevant, da die Hintergrundkarte statisch bleibt und Token-Bewegungen typischerweise sequentiell erfolgen – zu jedem Zeitpunkt bewegt sich meist nur ein einzelner Token.

Die konkreten Implementierungsdetails – einschließlich Texture-Management und Event-Handling – werden in Kapitel 3 im Kontext der Atlas-VTT-Entwicklung erläutert.

3 Konzeption und Implementierung

Dieses Kapitel beschreibt die Konzeption und Implementierung von Atlas VTT. Abschnitt 3.1 definiert zunächst die funktionalen und nicht-funktionalen Anforderungen, die sich aus etablierten VTT-Standards und der Obsidian-Plattform ableiten. ?? erläutert die architektonischen Entscheidungen und das Datenmodell. Abschließend dokumentiert ?? die Entwicklung zentraler Features mit besonderem Fokus auf die wissenschaftliche Begründung der getroffenen Designentscheidungen.

3.1 Anforderungsanalyse

Die Anforderungsanalyse fundiert die Konzeption und Implementierung von Atlas VTT. Die funktionalen und nicht-funktionalen Anforderungen werden aus bestehenden VTT-Standards, gängigen TTRPG-Standards und der Obsidian-Plattform abgeleitet und nach Priorität kategorisiert.

3.1.1 Funktionale Anforderungen

Wie bereits in Kapitel 2 erläutert digitalisieren VTTs Tabletop-RPG-Gameplay durch interaktive Karten, Token-basiertes Movement und Fog of War. Die funktionalen Anforderungen von Atlas VTT orientieren sich an etablierten VTT-Standards von Foundry VTT und Roll20 sowie an den Spielregeln von DnD 5e.

Die Fokussierung auf DnD 5e als Referenzsystem erfolgt aus zwei Gründen: Erstens dominiert DnD den TTRPG-Markt mit einem geschätzten Anteil von über 50% aller Verkäufe im Fachhandel [RPGDrop2024Market], wodurch die Implementierung der DnD 5e-Regeln die größte Zielgruppe adressiert. Zweitens erfordert die Entwicklung eines Minimum Viable Product (MVP) eine bewusste Einschränkung des Umfangs auf ein einzelnes Regelsystem, um innerhalb des Projektrahmens eine vollständige und nutzbare Implementierung zu gewährleisten. Die Architektur von Atlas VTT ist jedoch modular gestaltet, sodass zukünftige Erweiterungen für andere Regelsysteme möglich sind.

Im Gegensatz zu cloud-basierten Lösungen fokussiert Atlas VTT auf lokales Gameplay innerhalb Obsidian mit direkter Integration in das Vault-System.

Priorisierungskriterien Die Priorisierung der Anforderungen erfolgt nach der MoSCoW-Methode, einer etablierten Technik zur Anforderungspriorisierung im Requirements Engineering [IIBA2015BABOK]. MoSCoW kategorisiert Anforderungen in vier Stufen: *Must*

have (unverzichtbar), *Should have* (wichtig), *Could have* (wünschenswert) und *Won't have* (ausgeschlossen). Für eine fokussierte Darstellung werden bei Atlas VTT ausschließlich die Kategorien *Must have* und *Should have* verwendet.

Must have (M) umfasst Features, die in etablierten VTT-Lösungen als grundlegend gelten und ohne die grundlegendes VTT-Gameplay nicht möglich ist. Die Klassifikation als *Must have* erfolgt, wenn mindestens eines der folgenden Kriterien erfüllt ist: (1) Das Feature ist in allen marktführenden VTT-Lösungen (Foundry VTT, Roll20) als Kern-Feature implementiert [**FoundryVTT2024**, **Roll202024**], (2) das Feature ist für die Umsetzung grundlegender DnD 5e-Spielregeln erforderlich [**DnDBasicRules2014**], oder (3) das Feature ist technisch notwendig für die Persistenz und Session-Continuity (Sitzungskontinuität).

Should have (S) umfasst Features, die die Benutzererfahrung verbessern oder Obsidian-spezifische Integration bieten, allerdings nicht für grundlegendes VTT-Gameplay erforderlich sind. Diese Features werden oft als optionale Erweiterungen in etablierten VTTs angeboten oder dienen der Differenzierung von Atlas VTT durch vault-native Integration.

3.1.1.1 Karten-Management

Das Karten-Management bildet die Basis jeder VTT-Session. Atlas VTT muss verschiedene Bildformate (Joint Photographic Experts Group (JPEG), Portable Network Graphics (PNG), Web Picture format (WebP)) für Hintergrundkarten unterstützen und ein konfigurierbares Grid-Overlay bereitstellen. DnD 5e definiert das Grid-basierte Movement-System mit einer Standardskalierung, bei der jedes Quadrat auf dem Grid 5 Fuß repräsentiert [**DnDBasicRules2014**]. Die Grid-Konfiguration muss verschiedene Größen, Offsets und Typen (Square, Hexagonal) unterstützen. Battle Maps bezeichnen detaillierte Karten für taktische Kampfszenen, die typischerweise ein Grid-Overlay für präzise Token-Positionierung enthalten. Da diese Maps unterschiedliche Skalierungen aufweisen, ist eine flexible Grid-Konfiguration erforderlich.

Die Persistenz erfolgt durch ein benutzerdefiniertes `.atlasmap`-Dateiformat, das Map-Konfiguration und platzierte Objekte als JavaScript Object Notation (JSON) speichert. Dies ermöglicht Session-Continuity und Integration in Obsidians Vault-Struktur. Multi-Scene-Support ermöglicht den Wechsel zwischen verschiedenen Locations innerhalb einer Kampagne.

Die Anforderungen F1.1-F1.4 sind als *Must have* klassifiziert. Beide etablierten VTT-Plattformen implementieren F1.1-F1.3 als Kern-Features; ohne sie wäre grundlegendes taktisches Gameplay auf Basis von DnD 5e-Bewegungsregeln nicht möglich. Die Klassifikation basiert auf folgender Feature-Analyse:

Foundry VTT implementiert Grid-Konfiguration mit Unterstützung für Square, Hexagonal und Gridless Layouts sowie konfigurierbarer Grid-Größe und Offset-Anpassung [**FoundryScenes2024**]. Map-Import erfolgt über Drag & Drop mit Unterstützung für JPEG, PNG und WebP Formate. Die Scene-Konfiguration ermöglicht präzise Anpassung von Grid-

Typ, Größe und Ausrichtung, was für die korrekte Darstellung verschiedener Battle-Map-Formate erforderlich ist.

Roll20 bietet vergleichbare Funktionalität mit Snap-to-Grid für Square- und Hexagonal-Grids, wobei die Grid-Größe und der Typ in den Page Settings konfiguriert werden können [**Roll20Graphics2024**].

Die Persistenz-Anforderung (F1.4) ist technisch notwendig für Session-Continuity: Ohne Speicherung würden alle Map-Konfigurationen und platzierten Objekte bei jedem Plugin-Neustart verloren gehen. Multi-Scene-Support (F1.5) ist als Should have eingestuft, da sowohl Foundry als auch Roll20 zwar Scene-Management implementieren, einzelne Maps für minimale VTT-Funktionalität aber ausreichen und der Szenenwechsel primär Komfort-Funktionalität darstellt.

?? fasst die Karten-Management-Anforderungen zusammen:

Tabelle 3.1: Funktionale Anforderungen: Karten-Management

ID	Anforderung	MoSCoW	Standard
F1.1	Laden von Hintergrundkarten (JPEG, PNG, WebP)	M	VTT-Standard
F1.2	Grid-Overlay (Square, Hexagonal)	M	DnD 5e
F1.3	Grid-Konfiguration (Größe, Offset, Typ)	M	VTT-Standard
F1.4	Speicherung als .atlasmap Dateiformat	M	Persistenz
F1.5	Multi-Scene-Support	S	Nice-to-Have

3.1.1.2 Token-System

Tokens repräsentieren Entitäten in Form von Spieler- und Nicht-Spieler-Charakteren auf der Map und sind die zentralen interaktiven Objekte in einem VTT [**FoundryTokens2024**]. Die grundlegenden Anforderungen umfassen Token Creation & Placement, Drag & Drop Movement mit Grid-Snapping und Token Rotation. DnD 5e definiert für Movement-Berechnungen, dass eine typische mittelgroße Kreatur einen Raum von 5 Fuß Breite kontrolliert [**DnDBasicRules2014**], was in der Regel einer Token-Größe von 1×1 Grid-Squares entspricht.

Erweiterte Funktionen umfassen Multi-Selection für Gruppenbewegung, Health Bar Display für visuelles Hit Points (HP)-Tracking und Status Icons für Conditions wie Prone oder Stunned. Die Token-Statblock Linking-Funktion ermöglicht bidirektionale Verlinkung zwischen Tokens und Markdown-basierten Character Sheets im Obsidian Vault, wodurch ein Klick auf einen Token die zugehörigen Werte anzeigt. Diese Vault-native Integration unterscheidet Atlas VTT von cloud-basierten Lösungen.

Die Anforderungen F2.1-F2.5 sind als Must have klassifiziert, da sie die minimale Funktionalität für Token-basiertes Gameplay darstellen und in beiden etablierten VTT-Plattformen als Kern-Features implementiert sind:

Foundry VTT implementiert Token-Manipulation als Kern-Feature. Die Funktionen umfassen Token Creation durch Drag & Drop von Akteuren auf die Canvas, Placement mit präziser Positionierung über X/Y-Koordinaten sowie Movement mit automatischer Distanzmessung beim Ziehen. Für taktische Positionierung bietet das System Grid-Snapping sowie Rotation in Grad-Schritten mit Shift+WASD/Mausrad-Kontrolle [**FoundryTokens2024**]. Multi-Selection wird durch das Select Tokens Tool mit Shift-Click und Bounding-Box-Auswahl unterstützt.

Roll20 bietet vergleichbare Kern-Funktionalität: Drag & Drop von Tokens aus der Art Library auf den Tabletop, automatisches Grid-Snapping beim Movement (kann per Alt-Taste deaktiviert werden), sowie Token-Rotation mit 45°-Schritten für Square-Grids und 30°-Schritten für Hexagonal-Grids [**Roll20Graphics2024**, **Roll20TokenFeatures2024**]. Multi-Selection erfolgt durch Shift-Click oder Rahmenauswahl.

Token-Rotation (F2.4) wird in beiden Plattformen unterschiedlich granular implementiert (Foundry: beliebige Grad-Schritte, Roll20: feste Inkremente).

Die erweiterten Features F2.6-F2.8 (Health Bars, Status Icons, Statblock-Linking) sind als Should have eingestuft, da sie zwar die Benutzererfahrung verbessern könnten, aber nicht für grundlegendes Gameplay zwingend erforderlich sind. Health Bars und Status Icons werden in Foundry VTT als konfigurierbare Anzeigeelemente implementiert [**FoundryTokens2024**], während Roll20 diese als optionale Token-Eigenschaften anbietet [**Roll20TokenFeatures2024**]. Token-Statblock Linking (F2.8) ist eine Obsidian-spezifische Integration ohne direktes Äquivalent in cloud-basierten VTTs. ?? fasst die Token-System-Anforderungen zusammen.

Tabelle 3.2: Funktionale Anforderungen: Token-System

ID	Anforderung	MoSCoW	Standard
F2.1	Token Creation & Placement	M	VTT-Standard
F2.2	Drag & Drop Movement	M	VTT-Standard
F2.3	Grid-Snapping	M	DnD 5e
F2.4	Token Rotation	M	Combat Facing
F2.5	Token Selection (Single & Multi)	M	VTT-Standard
F2.6	Health Bar Display	S	Nice-to-Have
F2.7	Status Icons	S	Nice-to-Have
F2.8	Token-Statblock Linking	S	Obsidian-spezifisch

3.1.1.3 Fog of War

Wie in Abschnitt 2.1.1 beschrieben, ermöglicht Fog of War das Verbergen nicht-entdeckter Bereiche einer Map und ist ein Standard-Feature in VTTs wie Foundry VTT und Roll20 [FoundryScenes2024, Roll20FogOfWar2024]. Die Anforderungen umfassen das Erstellen von Fog Regions (Rectangle, Polygon), das progressive Aufdecken durch Fog Removal sowie separate Rendering-Modi: Der Spielleiter (Game Master (GM)) sieht die vollständige Karte zur Fog-Planung, während die Spieleransicht den Nebel undurchsichtig darstellt.

Alle Fog of War-Anforderungen (F3.1-F3.4) sind als Must have klassifiziert, da Fog of War in beiden etablierten VTT-Plattformen als Kern-Feature implementiert ist und ohne diese Funktionalität eine grundlegende Mechanik des Tabletop-Gameplays – das schrittweise Erkunden unbekannter Gebiete – nicht umsetzbar wäre:

Foundry VTT implementiert Fog of War als Vision-System. Die Fog Exploration wird pro Benutzer getrackt und in der Datenbank persistiert [FoundryScenes2024]. Das System unterscheidet zwischen GM-View (vollständige Map-Sichtbarkeit für Planung) und Token-basierter Player-Vision (limitiert auf aktuell sichtbare Bereiche). Fog wird automatisch aufgedeckt, wenn Tokens mit aktivierter Vision durch Bereiche bewegen.

Roll20 bietet Advanced Fog of War, das grid-basierte Fog Cells verwendet: Wenn ein Token eine Grid-Zelle passiert, wird diese dauerhaft aufgedeckt und für den kontrollierenden Spieler sichtbar [Roll20FogOfWar2024]. Das System unterstützt sowohl Rectangle- als auch Polygon-Tools für manuelle Fog-Manipulation. Die Fog-Aufdeckung wird pro Token gespeichert, wodurch jeder Spieler eine individuelle Map-Exploration haben kann. Roll20 implementiert separate Darstellungsmodi für Spielleiter (vollständige Kartensicht) und Spieler (undurchsichtiger Nebel).

Die Unterscheidung zwischen GM- und Player-View (F3.3, F3.4) ist technisch notwendig, um dem Spielleiter Map-Vorbereitung und Fog-Planung zu ermöglichen, ohne unentdeckte Bereiche für Spieler zu spoilern. Beide Plattformen implementieren diese Separation als Kern-Feature ihrer Fog-Systeme. ?? fasst die Fog of War-Anforderungen zusammen.

Tabelle 3.3: Funktionale Anforderungen: Fog of War

ID	Anforderung	MoSCoW	Standard
F3.1	Fog Region Creation (Rectangle, Polygon)	M	VTT-Standard
F3.2	Fog Removal/Reveal	M	VTT-Standard
F3.3	GM-Only Preview (GM sieht full map)	M	VTT-Standard
F3.4	Player View (Fog opaque)	M	VTT-Standard

3.1.1.4 Werkzeuge und Interaktion

Interaktive Werkzeuge unterstützen taktisches Gameplay und Map-Annotation. Das Measure Tool ermöglicht Distanzmessungen zwischen Entitäten auf der Map. DnD 5e definiert standardisierte Reichweiten für bestimmte Spielmechaniken, die in einem Virtual-Tabletop-System nachvollzogen werden müssen [DnDBasicRules2014]. Note Pins verlinken Map-Positionen mit beliebigen Markdown-Dateien im Vault, wodurch VTT-Inhalte mit bestehenden Notizen und Daten anderer Plugins (z.B. Dataview) verknüpft werden können – diese Obsidian-spezifische Integration unterscheidet Atlas VTT von anderen VTT-Lösungen. Drawing Tools und Text Placement ermöglichen zusätzliche Map-Annotationen.

Das Measure Tool (F4.1) ist als Must have klassifiziert, da Distanzmessung für die Umsetzung der DnD 5e-Spielregeln erforderlich ist: Bewegungsreichweiten, Zauberreichweiten und Waffenreichweiten sind in Fuß definiert [DnDBasicRules2014], was präzise Grid-basierte Messung erfordert. Die Implementierung in beiden etablierten VTT-Plattformen als Kern-Feature bestätigt diese Einstufung:

Foundry VTT implementiert ein Distance Measurement Ruler Tool (aktivierbar per R-Taste), das Wegpunkt-Unterstützung bietet: Ctrl+Click ermöglicht das Setzen mehrerer Wegpunkte für komplexe Messungen oder Elevation-Berücksichtigungen. Das Tool unterscheidet zwischen Distance Measurement (reine Entfernung) und Token Drag Measurement (Movement Cost unter Berücksichtigung von schwierigem Terrain). Templates für Area-of-Effect-Berechnungen werden ebenfalls unterstützt (Circle, Cone, Rectangle, Ray) [FoundryMeasurement2024].

Roll20 bietet ein Measurement Tool, das bei aktiviertem Grid automatisch Grid-basierte Distanzen berechnet. Das Tool unterstützt Wegpunkt-basierte Messungen für nicht-lineare Pfade und berücksichtigt Grid-Typ (Square vs. Hexagonal) bei der Distanzberechnung [Roll20Graphics2024].

Die erweiterten Features F4.2-F4.4 (Drawing Tools, Note Pins, Text Placement) sind als Should have eingestuft, da sie optionale Annotations-Features darstellen, die zwar die Benutzererfahrung verbessern könnten, aber nicht für regelkonformes DnD-Gameplay erforderlich sind. Beide Plattformen implementieren Drawing-Funktionalität als zusätzliche Features, jedoch nicht als Must have. Note Pins (F4.3) bieten Obsidian-spezifische Integration ohne direktes Äquivalent in cloud-basierten VTTs und stellen eine Differenzierung von Atlas VTT dar. ?? fasst die Werkzeug-Anforderungen zusammen.

3.1.1.5 Character Management

Der Statblock Builder ermöglicht die Erstellung von Character Sheets direkt in Obsidian. Statblocks werden als Markdown-Dateien mit YAML Ain't Markup Language (YAML) Frontmatter im Vault gespeichert, wodurch sie mit Obsidians Linking-System kompatibel sind. Die Token-Statblock Linking-Funktion ermöglicht bidirektionale Verlinkung zwischen

Tabelle 3.4: Funktionale Anforderungen: Werkzeuge

ID	Anforderung	MoSCoW	Standard
F4.1	Measure Tool (Distance Measurement)	M	DnD 5e
F4.2	Drawing Tools (Freehand, Shapes)	S	Nice-to-Have
F4.3	Note Pins (Link zu Obsidian Notes)	S	Obsidian-spezifisch
F4.4	Text Placement	S	Nice-to-Have

Map-Tokens und Character Sheets. Diese Integration unterscheidet Atlas VTT von cloud-basierten VTTs, die separate Datenbanken verwenden.

3.1.2 Nicht-funktionale Anforderungen

Die nicht-funktionalen Anforderungen legen messbare Qualitätsattribute für Performance, Skalierbarkeit, Plattform-Kompatibilität und Wartbarkeit fest. Diese Anforderungen sind quantifizierbar und basieren auf Industriestandards sowie Plattform-Constraints.

3.1.2.1 Performance-Anforderungen

Performance stellt einen zentralen Faktor für flüssige Interaktionen in Desktop-Anwendungen dar. Nielsen beschreibt, dass Nutzer bei einer Reaktionszeit von 0,1 Sekunden (100 milliseconds (ms)) das System als augenblicklich reagierend wahrnehmen [Nielsen1993ResponseTimes]. Für kontinuierliche Interaktionen wie Maus-basiertes Dragging liegt die Wahrnehmungsschwelle jedoch niedriger: Forch et al. ermittelten in einer empirischen Studie einen Schwellenwert von 60 ms [Forch2017LatencyPerception]. Für Animationen und kontinuierliches Rendering zeigen Studien, dass Performance-Verbesserungen bei etwa 60 FPS ein Plateau erreichen [Janzen2014FrameRate], weshalb dieser Wert als Ziel für hochwertige interaktive Anwendungen gilt. 30 FPS stellt hingegen das Minimum für flüssige Interaktionen dar, da niedrigere Frame-Raten zu signifikanten Performance-Einbußen führen [Janzen2014FrameRate].

VTTs müssen große Maps und viele Tokens performant handhaben. Die PIXI.js-Dokumentation betont, dass die Szenen-Komplexität direkt die Performance beeinflusst und das System mit zunehmender Objektanzahl langsamer wird [PixiPerf2024], was bei VTT-Szenarien mit 50 bis 100 Tokens relevant ist. Obsidian basiert auf Electron, dessen Arbeitsspeicher pro Prozess auf maximal 4 GB begrenzt ist [ElectronV8MemoryCage2022]. Da alle Plugins im selben Renderer-Prozess laufen und sich dieses Limit teilen, wurde für Atlas VTT ein konservatives Design-Ziel von 500 Megabyte (MB) für typische Szenarien festgelegt.

?? definiert die quantifizierten Performance-Ziele:

Tabelle 3.5: Nicht-funktionale Anforderungen: Performance

ID	Metrik	Ziel	Testszenario	Standard
NF1.1	Frame Rate	≥ 30 FPS	50 Tokens, 2048px Map	Janzen et al.[Janzen2014FrameRate]
NF1.2	Frame Rate (Ideal)	≥ 60 FPS	Standard-Szenarien	Janzen et al.[Janzen2014FrameRate]
NF1.3	Interaction Latency	< 60 ms	Drag & Drop Token	Forch et al.[Forch2017LatencyPerception]
NF1.4	Map Load Time	< 2 s	4096px Map	UX-Richtlinien
NF1.5	Memory Footprint	< 500 MB	100 Tokens, 10 Maps	Electron

Standardisierte Test-Szenarien validieren diese Anforderungen, wie ?? detailliert beschreibt. Diese Methodik gewährleistet die wissenschaftliche Reproduzierbarkeit der Performance-Metriken.

3.1.2.2 Skalierbarkeit

Skalierbarkeits-Anforderungen definieren die Grenzen für Token Count, Map Size und Asset Library. Stress-Test-Szenarien mit mehr als 100 Tokens bei mindestens 30 FPS simulieren große DnD-Encounters. Für die Performance-Validierung dient eine Testauflösung von 4096×4096 Pixel. Die technische Strategie zur Erreichung dieser Skalierbarkeit basiert auf etablierten Performance Best Practices: Texture Caching für wiederverwendete Assets [**PixiPerf2024**] sowie Power-of-Two Textures für GPU-Optimierung [**PixiTexture2024**]. Die PIXI.js-Dokumentation empfiehlt zusätzlich Culling-Strategien für komplexe Szenen [**PixiPerf2024**], die im aktuellen Entwicklungsstand allerdings nicht implementiert sind.

3.1.2.3 Plattform-Kompatibilität

Obsidian-Plugins unterliegen spezifischen Plattform-Constraints, die Architektur-Entscheidungen beeinflussen. Das Single-Bundle-Constraint erfordert, dass alle Plugin-Funktionalität in einer main.js Datei gebündelt wird, was Code-Splitting unmöglich macht und Tree-Shaking sowie Bundle Size Monitoring kritisch werden lässt [**ObsidianPluginDev2024**]. Das empfohlene Limit von 244 Kilobyte (KB) wird von großen Plugins regelmäßig überschritten; das Ziel für Atlas VTT liegt bei maximal 3 MB.

Ein kritischer Konflikt entsteht durch Obsidians globales PIXI.js v7 Bundle. Atlas VTT benötigt PIXI.js v8 für moderne Features, was explizite v8 Imports via npm erfordert, um Namespace-Kollisionen zu vermeiden. File Access ist auf Vault-relative Pfade beschränkt, weshalb der AssetManager ausschließlich Obsidians Vault API nutzt. Tailwind CSS ist nur

eingeschränkt nutzbar, wodurch Styling primär durch Sassy CSS (SCSS) und Obsidian Native Styles erfolgt. ?? fasst die Plattform-Constraints und deren Mitigationsstrategien zusammen.

Tabelle 3.6: Nicht-funktionale Anforderungen: Plattform-Constraints

ID	Constraint	Impact	Mitigation
NF3.1	Single-Bundle	Kein Code-Splitting	Tree-Shaking, Monitoring
NF3.2	PIXI.js v7 Global	Namespace Collision	Explizite v8 Imports
NF3.3	File Access	Vault-relative Paths	AssetManager + Vault API
NF3.4	Tailwind Limited	Styling Constraints	SCSS + Native Styles

3.1.2.4 Wartbarkeit und Code-Qualität

Etablierte Software Engineering Prinzipien gewährleisten Wartbarkeit. Die modulare Service-Architektur setzt das Single Responsibility Principle um, wobei jedes Modul maximal 300 Lines of Code umfassen sollte [**DigitalOceanSOLID, ESLintMaxLines2024**]. Strict TypeScript mit vollständiger Type Safety eliminiert any-Types in Production Code. Das DRY-Prinzip begrenzt Code Duplication auf unter 5%.

Die Architektur folgt Service-Oriented Architecture mit Event-Driven Communication für Loose Coupling und Singleton State durch Zustand Store in `main.ts:atlasStore`. Code-Qualität Metriken umfassen Cyclomatic Complexity unter 10 pro Funktion, Function Length unter 50 Lines of Code und File Length unter 300 Lines of Code als Refactoring Threshold. Diese Metriken werden kontinuierlich überwacht, um Code-Wartbarkeit über den gesamten Entwicklungszyklus sicherzustellen.

3.1.2.5 Benutzerfreundlichkeit

Usability-Anforderungen fokussieren auf eine benutzerfreundliche User Interface (UI), Keyboard Shortcuts für erfahrene Benutzer, Drag & Drop für Standard-VTT-Interaktionen und Undo/Redo (Ctrl+Z/Ctrl+Y) für Fehlerkorrektur. Die Implementierung nutzt Zustand mit Zustand für Undo/Redo-Funktionalität und eine Toolbar mit Tool-Modus-Switching. Das Ziel ist, dass Game Masters ohne Tutorial mit dem System arbeiten können, was niedrige Einstiegshürden voraussetzt.

3.2 Systemdesign

Auf Basis der definierten Anforderungen beschreibt dieser Abschnitt das Systemdesign von Atlas VTT. ?? erläutert die architektonischen Entscheidungen, die Layer-Struktur und die

Integration mit der Obsidian Plugin API. ?? beschreibt das Datenmodell mit den Persistierungsstrategien für Map-Daten, Asset-Metadaten und Application State.

3.2.1 Architekturentwurf

Die Architektur von Atlas VTT folgt einem modularen, service-orientierten Ansatz, um die spezifischen Herausforderungen eines VTT-Plugins in der Obsidian-Umgebung zu adressieren. Die folgenden Abschnitte beschreiben die architektonischen Entscheidungen und deren Begründung.

3.2.1.1 Design-Prinzipien

Die Architektur von Atlas VTT basiert auf vier Software-Engineering-Patterns. Eine **service-orientierte Architektur (Service-Oriented Architecture (SOA))** organisiert die Funktionalität in verschiedene eigenständige Services mit jeweils klar definierter Verantwortlichkeit [Fowler2014]. Services wie AssetService, NetworkService und TokenStatblockLinkService können dadurch unabhängig entwickelt und getestet werden.

Das **Single Responsibility Principle (Single Responsibility Principle (SRP))** stellt sicher, dass jedes Modul nur eine Änderungsursache hat [DigitalOceanSOLID]. Dies reduziert die Kopplung zwischen Komponenten und erhöht die Kohäsion innerhalb einzelner Module. Eng damit verbunden ist die **Event-Driven Communication**, die Services durch asynchrone Event-Emission entkoppelt. Dieses Pattern entspricht dem Observer Pattern [RefactoringGuruObserver]: Services kommunizieren über Events wie atlas-vtt:refresh-assets, wodurch neue Features ohne Modifikation bestehender Services hinzugefügt werden können.

Für Services mit vault-weitem Zustand – insbesondere den AssetService – kommt das **Singleton Pattern** zum Einsatz [RefactoringGuruSingleton]. Asset-Metadaten müssen über alle geöffneten Map-Views hinweg konsistent sein; multiple Instanzen würden zu Dateninkonsistenzen führen.

Der Trade-off dieser Architektur liegt in erhöhter Code-Komplexität durch Service-Registration, Event-Handling und Dependency Injection [Fowler2014]. Bei einem Plugin mit mehr als 30.000 Zeilen Code überwiegen dennoch die Vorteile in Wartbarkeit und Testbarkeit.

3.2.1.2 Layer-Architektur

Die Implementierung folgt einer Layered Architecture, bei der Komponenten in horizontale Schichten mit spezifischen Rollen strukturiert sind [Richards2024MicrokernelArchitecture]. Atlas VTT implementiert vier Haupt-Layer.

View Layer Der View Layer umfasst alle Benutzeroberflächen-Komponenten. Die `atlas-view.ts` stellt die GM/Host View für Spielleiter bereit, während `player-view.ts` ein dediziertes Player Window ohne GM-spezifische Elemente implementiert. Ergänzt werden diese durch die `dashboard-view.tsx` für Session-Management und die `statblock-view.tsx` für Charakterbögen im DnD 5e Layout. Alle Views sind als React 19 Komponenten implementiert und nutzen UI-Komponenten aus dem Radix UI Ecosystem.

Rendering Layer Der Rendering Layer stellt Map, Tokens und Effekte auf einem HTML5 Canvas dar. Der `PixiRendererOrchestrator.ts` orchestriert den Rendering-Prozess, während der `LayerManager.ts` die Rendering-Layer (Background, Grid, Tokens, Fog, UI) verwaltet. Für Kamera-Kontrolle mit Pan, Zoom und Boundary-Handling kommt `pixi-viewport` zum Einsatz.

Die strikte Separation zwischen React (UI Layer) und PIXI.js (Rendering Layer) stellt eine architektonisch relevante Entscheidung dar: React-Komponenten modifizieren niemals direkt PIXI.js Display Objects, sondern aktualisieren den zentralen State. Renderer reflektieren anschließend den neuen State. Diese unidirektionale Datenfluss-Architektur entspricht dem Flux-Pattern, einem von Facebook entwickelten Architekturmuster für vorhersagbaren Datenfluss, und verhindert Race Conditions zwischen UI-Updates und Canvas-Rendering.

State Management Layer Der State Management Layer implementiert eine Centralized-State-Architecture mit Zustand. Der globale Application State in `atlasStore.ts` umfasst Map-Daten, UI-Zustand und Tool-Modus. Eine `storeFactory.ts` erzeugt View-spezifische Store-Instanzen, während `tabStore.ts` Tab-spezifischen State für Multi-Map-Support verwaltet. Die Middlewares `zundo` und `immer` ermöglichen Undo/Redo-Funktionalität und immutable State-Updates.

Die Verwendung von Immer garantiert Immutability durch strukturelles Sharing: State-Updates erzeugen neue Objekte statt bestehende zu mutieren. Der Copy-on-Write-Algorithmus kopiert nur geänderte Teile des State-Trees, wodurch der Performance-Overhead minimiert wird [ImmerDocs2024].

Service Layer Der Service Layer enthält mehr als 50 spezialisierte Services für Geschäftslogik, externe API-Calls und Daten-Persistierung. Der `AssetService` verwaltet als Singleton vault-weite Asset-Metadaten, der `TokenStatblockLinkService` implementiert die bidirektionale Token-Statblock-Verlinkung. Weitere Services umfassen den `NetworkService` für WebSocket-basiertes Multiplayer, den `AudioService` für Sound Effects und den `MapService` für Map Loading und Validation.

Das Singleton Pattern beim `AssetService` adressiert eine wesentliche Anforderung: Die Verlinkung eines Tokens zu einem Statblock muss über alle Maps hinweg konsistent sein. Multiple Instanzen würden zu inkonsistenten Metadaten-Zuständen führen, wenn zwei Maps gleichzeitig dieselben Assets verwenden.

```
// Falsch: Direkter Zugriff kann fehlschlagen
const view = this.app.workspace.getLeavesOfType("atlas-vtt")[0].view;

// Korrekt: Explizites Laden vor Zugriff
const leaf = this.app.workspace.getLeavesOfType("atlas-vtt")[0];
await leaf.loadIfDeferred();
const view = leaf.view as AtlasView;
```

Listing 3.1: DeferredView Safety Pattern

3.2.1.3 Integration mit Obsidian Plugin API

Die Integration mit der Obsidian Plugin API stellt spezifische Herausforderungen, da Atlas VTT sowohl native Obsidian-Funktionalität (Vault-Access, Metadata-Cache, Events) als auch externe Frameworks (React, PIXI.js) kombinieren muss.

Plugin Lifecycle Management Atlas VTT integriert sich über die Standard Plugin Lifecycle Hooks in Obsidians Plugin-System. Die `onload()`-Methode initialisiert das Plugin in vier Schritten: (1) Registrierung der Custom View Types für die Atlas-UI, (2) Initialisierung der Singleton Services (AssetService, TokenStatblockLinkService), (3) Registrierung von Commands und Ribbon Icons für die Obsidian-UI, und (4) Event-Handler-Setup für Workspace-Events. Die `onunload()`-Methode führt Cleanup durch und entfernt alle Atlas-Views aus dem Workspace, um Memory Leaks zu vermeiden (vollständige Implementierung: siehe ??, `main.ts:45-67`).

DeferredView Handling Ein kritischer Obsidian-spezifischer Constraint ist das DeferredView-System: Obsidian lädt Views lazy, um Startup-Performance zu optimieren. Direkter Zugriff auf eine nicht-geladene View führt zu undefined-Fehlern. ?? zeigt das Safety Pattern für explizites Laden vor Zugriff.

Dieser Constraint stellt ein dokumentiertes Risiko dar und erfordert explizites Laden via `await leaf.loadIfDeferred()` vor jedem View-Zugriff.

PIXI.js Version Konflikt Eine kritische Integration-Herausforderung entsteht durch Obsidians globales PIXI.js v7 Bundle. Obsidian stellt PIXI.js v7 als `window.PIXI` bereit, Atlas VTT benötigt hingegen PIXI.js v8 für v8-spezifische Features wie den optimierten Batch Renderer und WebGPU-Vorbereitung. Die Verwendung beider Versionen gleichzeitig würde zu Type-Konflikten und Runtime-Fehlern führen.

Die Lösung besteht darin, PIXI.js v8 als explizite npm-Dependency zu deklarieren und ausschließlich über ES6-Imports zu verwenden, wie ?? demonstriert.

Diese Entscheidung unterscheidet sich von Foundry VTT, das bei PIXI.js v7 verbleibt. Das Foundry-Entwicklerteam begründet dies damit, dass eine Migration Breaking Chan-

```
// Falsch: Verwendet Obsidians PIXI v7
const app = new PIXI.Application();

// Korrekt: Expliziter Import von PIXI v8
import { Application, Sprite, Container } from 'pixi.js';
const app = new Application();
```

Listing 3.2: PIXI.js v8 Import Pattern

ges für das Modul-Ökosystem bedeuten würde [FoundryIssue11183]. Da Atlas VTT als vollständige Neuentwicklung ohne bestehende Codebasis entsteht, entfallen Migrationskosten. Gleichzeitig können v8-Features wie das vereinfachte Package-System und asynchrone Initialisierung für WebGPU-Support genutzt werden [PixiV8Migration2024].

Der Trade-off dieser Lösung ist erhöhte Bundle-Size: PIXI.js v8 fügt 500 KB zum finalen Bundle hinzu. Dieser Overhead wird bewusst in Kauf genommen, da die konsolidierte Package-Struktur von v8 Version-Konflikte eliminiert und die modernen API-Patterns die Wartbarkeit erhöhen [PixiV8Migration2024].

3.2.2 Datenmodell

Das Datenmodell von Atlas VTT adressiert VTT-spezifische Anforderungen wie Multi-Scene-Support, Undo/Redo und vault-wide Asset-Konsistenz. Die folgenden Abschnitte beschreiben die Datenstrukturen sowie deren Persistierungsstrategien und Begründung der Design-Entscheidungen.

3.2.2.1 Datenmodell-Architektur

Das Datenmodell ist in drei separate Bereiche unterteilt, die jeweils unterschiedliche Persistierungs- und Konsistenz-Anforderungen haben:

Map Data (persistent, pro-Map) speichert Map-spezifische Informationen wie Background-Image, Grid-Konfiguration und platzierte Objekte im `.atlasmap` JSON-Format. Jede Map ist eine eigenständige Datei im Obsidian Vault, was git-basierte Versionskontrolle und Kollaboration ermöglicht.

Asset Metadata (persistent, vault-wide) verwaltet wiederverwendbare Assets wie Token-Images, Sound Effects und Map-Templates in einer gemeinsamen `assets-metadata.json` Datei. Diese Separation verhindert Daten-Duplikation: Ein Token-Image kann in mehreren Maps verwendet werden, ohne dass dessen Metadaten (Tags, Statblock-Link) redundant gespeichert werden müssen.

Application State (transient, pro-View) hält Runtime-Zustand wie aktuelle Tool-Auswahl, Viewport-Position und UI-Zustand. Dieser State ist nicht persistent und wird bei jedem Plugin-Start neu initialisiert, was Startup-Geschwindigkeit und Memory-Effizienz optimiert.

Diese Separation entspricht dem Table Data Gateway Pattern [FowlerTableDataGateway2024], bei dem Daten-Zugriff von Geschäftslogik getrennt wird. Für Atlas VTT bedeutet dies, dass Services ausschließlich über definierte Schnittstellen (MapService, AssetService) auf persistente Daten zugreifen, nicht direkt auf Dateien.

3.2.2.2 Map Data Format

Das `.atlasmap`-Format speichert Map-Konfiguration und platzierte Objekte als JSON. Die Datenstruktur umfasst ein `version`-Feld für Schema-Versionierung, `name` und `background`-Konfiguration (Image-Pfad und Grid-Parameter), sowie ein `objects`-Array mit platzierten Map-Objekten (Token, FogRegion, etc.). Jedes Objekt enthält `type`, `id`, Position (`x/y`) und typ-spezifische Properties wie `statblockPath` für Tokens oder `shape/points` für FogRegions (vollständiges Schema: siehe ??).

Mehrere Faktoren motivierten die Wahl von JSON als Datenformat: JSON ist human-readable und damit git-friendly, was Diff-Visualisierung und Merge-Konflikt-Auflösung bei kollaborativer Map-Entwicklung erleichtert [JSON2017]. Binäre Formate wie Protocol Buffers wären kompakter (30% kleinere Files), jedoch auf Kosten der Lesbarkeit und Git-Integration. Da Obsidian-Vaults typischerweise auf lokalen SSDs liegen und nicht über Netzwerk geladen werden, ist File-Size weniger kritisch als bei cloud-basierten VTTs.

Das `objects`-Array enthält polymorphe Objekte verschiedener Typen (Token, FogRegion, NotePin), die durch ein `type`-Diskriminator-Feld unterschieden werden. Dieses Pattern entspricht dem Subtype Polymorphism Pattern und ermöglicht Erweiterbarkeit: Neue Objekt-Typen können hinzugefügt werden, ohne bestehende Maps zu invalidieren, solange das `version`-Feld entsprechend inkrementiert wird.

MapObject-Hierarchie Die TypeScript-Implementierung nutzt eine abstrakte Basisklasse `MapObject` für gemeinsame Eigenschaften aller Map-Objekte (`id`, `x`, `y`, `gmOnly`, `draggable`). Konkrete Klassen wie `Token` und `FogRegion` erben von dieser Basisklasse und fügen typ-spezifische Properties hinzu (`image`, `rotation`, `statblockPath` für Tokens; `shape`, `points` für FogRegions). Jede Klasse implementiert eine `serialize()`-Methode, die das Objekt in JSON-kompatibles Format konvertiert und ein `type`-Diskriminator-Feld hinzufügt (vollständige Implementierung: siehe ??, `types.ts:89-125`).

Diese Hierarchie folgt dem Open/Closed Principle: Das System ist offen für Erweiterung (neue Objekt-Typen) aber geschlossen für Modifikation (bestehende Typen bleiben unverändert) [StackifyOCP]. Der Trade-off liegt in erhöhter Type-Komplexität: TypeScript's Type System benötigt Type Guards für Discriminated Unions, um Type-Safety zur Compile-Zeit zu garantieren.

```

{
  "tokens": [
    {
      "id": "uuid-token-001",
      "name": "Goblin",
      "imagePath": "atlas-vtt/tokens/goblin.png",
      "statblockPath": "statblocks/Goblin.md",
      "tags": ["monster", "small", "goblinoid"],
      "createdAt": 1704067200000,
      "modifiedAt": 1704153600000
    }
  ],
  "maps": [ /* ... */ ],
  "sounds": [ /* ... */ ]
}

```

Listing 3.3: Asset Metadata Schema

3.2.2.3 Asset Metadata System

Asset-Metadaten werden vault-wide in `assets-metadata.json` gespeichert, um Konsistenz über alle Maps hinweg zu garantieren. ?? zeigt das Schema dieser übergreifenden Metadaten-Datei.

Eine zentrale Anforderung motivierte die Entscheidung für eine zentrale Metadata-Datei statt verteilter Metadaten (z.B. Sidecar-Files pro Asset): Asset-Metadaten müssen vault-weit konsistent sein, sodass die Verlinkung eines Tokens zu einem Statblock über alle Maps hinweg identisch bleibt. Eine dezentrale Lösung würde bei Änderungen (z.B. Statblock-Link-Update) alle Maps durchsuchen und aktualisieren müssen, was bei Vaults mit mehr als 50 Maps zu messbaren Performance-Einbußen führen würde, da jede Änderung alle Map-Dateien lesen und schreiben müsste. Die zentrale Lösung ermöglicht direkten Zugriff auf Metadaten über `AssetService` als Singleton, ohne dass Maps durchsucht werden müssen.

Der Trade-off ist Skalierbarkeit: Da alle Metadaten beim Laden in den Arbeitsspeicher gelesen werden, wächst der Memory-Footprint linear mit der Asset-Anzahl. Für die in Abschnitt 3.1 definierten Szenarien (1000 Assets) beträgt dieser Overhead weniger als 1 MB, während die Texture-Daten derselben Assets mehrere hundert MB beanspruchen.

Singleton AssetService Der `AssetService` implementiert das Singleton Pattern [RefactoringGuruSingleton], um Metadata-Konsistenz bei mehreren gleichzeitig geöffneten Maps zu garantieren. Die Implementierung verwendet eine private statische instance-Variable und einen privaten Constructor, der verhindert, dass externe Klassen direkt Instanzen erzeugen. Die statische `getInstance()`-Methode prüft, ob bereits eine Instanz existiert, und erstellt nur bei Bedarf eine neue. Die `getAssets()`-Methode lädt

```

---
name: "Goblin"
token-image: "atlas-vtt/tokens/goblin.png"
hp: 7
ac: 15
speed: 30
abilities:
  str: 8
  dex: 14
  con: 10
  int: 10
  wis: 8
  cha: 8
traits:
  - name: "Nimble Escape"
    description: "Can Disengage or Hide as bonus action"
actions:
  - name: "Scimitar"
    description: "+4 to hit, 1d6+2 slashing damage"
---

# Goblin

Additional GM notes...

```

Listing 3.4: Statblock YAML Frontmatter

Metadata lazy beim ersten Zugriff und cached sie für nachfolgende Aufrufe (vollständige Implementierung: siehe ??, `AssetService.ts:23-45`).

Diese Implementierung garantiert, dass alle Map-Views denselben `AssetService` verwenden und somit konsistente Metadaten sehen. Ohne Singleton würde View A Asset-Metadaten ändern, während View B veraltete Daten hat, was zu Race Conditions beim gleichzeitigen Token-Spawning führen würde.

3.2.2.4 Statblock Data

Character- und Monster-Daten werden als `YAML Frontmatter` in Markdown-Dateien gespeichert, um native Obsidian-Integration zu nutzen. ?? zeigt ein Beispiel für einen `DnD 5e` Statblock.

Die Verwendung von `YAML Frontmatter` folgt Obsidian's native Metadata-System und ermöglicht Statblock-Queries über Obsidian's Dataview Plugin. Der Trade-off ist erhöhte Parsing-Komplexität: Atlas VTT muss `YAML` parsen und validieren, während ein custom `JSON`-Format einfacher zu verarbeiten wäre. Der Vorteil liegt in der Interoperabilität: Statblocks sind als normale Markdown-Notes mit beliebigen anderen Vault-Dateien und Plugins wie Dataview oder Templater kompatibel.

Bidirektionale Token-Statblock-Verlinkung Das System erzwingt One-to-One Relationships zwischen Tokens und Statblocks: Ein Token kann maximal einen Statblock referenzieren, und ein Statblock kann maximal ein Token haben. Dies wird durch den `TokenStatblockLinkService` (Singleton) garantiert. ?? illustriert den Ablauf der Verlinkungslogik:

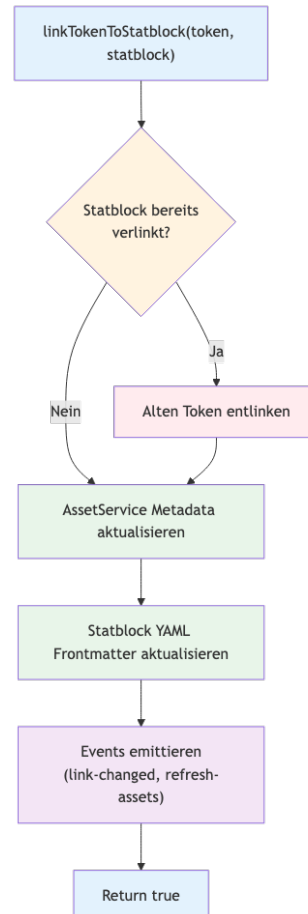


Abbildung 3.1: Ablaufdiagramm der bidirektionalen Token-Statblock-Verlinkung. Der Prozess stellt durch Konsistenz-Checks sicher, dass One-to-One Relationships erzwungen werden (vollständiger Quellcode: siehe ??).

Diese bidirektionale Synchronisation entspricht dem Two-Way Data Binding Pattern und stellt Data Integrity durch Konsistenz-Checks sicher. Der Performance-Overhead ist minimal, da nur die betroffene Metadata-Datei und das spezifische `YAML`-Frontmatter aktualisiert werden müssen, ohne dass andere Dateien durchsucht werden (Implementierung: `TokenStatblockLinkService.ts:67-95`).

3.2.2.5 Application State Management

Der Runtime-Zustand wird mit Zustand [ZustandDocs2024] als State Management Library verwaltet. ?? zeigt die mehrschichtige Architektur und den zyklischen Datenfluss des Atlas Stores.

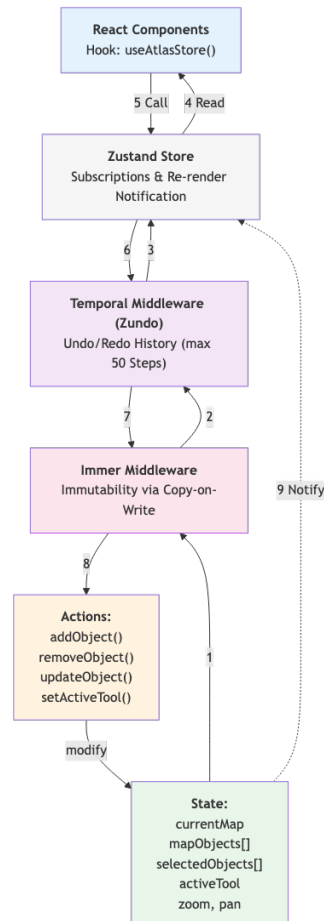


Abbildung 3.2: Architektur des Atlas State Management Systems (vollständige Implementierung: siehe ??).

Die Architektur unterscheidet drei Pfade: Der **Read-Pfad** (1–4, blau) ermöglicht Components das Lesen des States durch einen Middleware-Layer. Der **Write-Pfad** (5–8, rot) verarbeitet Actions, die den State modifizieren. Der **Notify-Pfad** (9, grün) benachrichtigt Subscribers über Änderungen. Temporal fügt Undo/Redo-Funktionalität hinzu, während Immer Immutability erzwingt.

```
// src/app/MapController.ts:234-256
private setupAutosave() {
  let timeout: NodeJS.Timeout;

  this.store.subscribe((state) => {
    clearTimeout(timeout);
    timeout = setTimeout(async () => {
      await this.saveMap(state.currentMap);
    }, 1000); // 1 second debounce
  });
}

private async saveMap(mapData: MapData) {
  const json = JSON.stringify(mapData, null, 2);
  await this.app.vault.modify(this.mapFile, json);
}
```

Listing 3.5: Autosave with Debouncing

Die Verwendung von Immer garantiert Immutability durch Copy-on-Write: State-Updates erzeugen neue Objekte statt Mutations, was Time-Travel Debugging und Undo/Redo ohne zusätzliche Logik ermöglicht [ImmerDocs2024]. Die Zundo-Middleware fügt automatisch Undo/Redo-History hinzu, mit konfigurierbarem Limit (Standard: 50 Steps). Der vollständige Code des AtlasState-Interfaces und der Store-Konfiguration ist in ?? zu finden (Implementierung: atlasStore.ts:34-67).

Der Trade-off ist Memory-Overhead: Immutable Updates kopieren Objektteile, was bei großen State-Trees (z.B. mehr als 100 MapObjects) zu erhöhtem Memory-Verbrauch führt. Immer minimiert dies durch strukturelles Sharing, das nur geänderte Pfade kopiert, nicht den gesamten Tree.

3.2.2.6 Datenpersistenz und Synchronisation

Die Persistierung von Map-Änderungen nutzt Debouncing, um Input/Output (I/O)-Operationen zu reduzieren. Debouncing verwirft Operationen, die innerhalb eines definierten Zeitintervalls zu dicht aufeinander folgen, und konsolidiert sie zu einem einzigen Aufruf [MDN2024Debounce]. ?? zeigt die Implementierung.

Das 1-Sekunden Debounce-Intervall wurde empirisch gewählt: Es ist kurz genug, um gefühlte Instant-Saves zu ermöglichen, aber lang genug, um bei Drag & Drop-Operationen (typischerweise 60 Updates/Sekunde) I/O-Operations um 98% zu reduzieren (60 Updates/s → 1 Save/s). Ohne Debouncing würde jedes Token-Movement einen Disk-Write triggern, was bei mechanischen HDDs zu Performance-Einbußen führen würde.

Event-Driven Multi-View Synchronisation Bei mehreren gleichzeitig geöffneten Maps kommunizieren Views über Obsidian's Workspace Events. ?? illustriert dieses Pattern.

```

// src/app/services/TokenStatblockLinkService.ts:178-195
async linkTokenToStatblock(...) {
  // ... perform linking logic

  // Emit custom event for Service-to-Service communication
  this.emit("link-changed", { tokenImagePath, statblockPath });

  // Emit workspace event for View-to-View communication
  this.app.workspace.trigger("atlas-vtt:refresh-assets");

  // All views listen for this event and refresh their asset data
}

// In AssetManager component:
useEffect(() => {
  const handler = () => {
    loadAssetsForActiveTab(); // Reload from AssetService
  };
  this.app.workspace.on("atlas-vtt:refresh-assets", handler);
  return () => this.app.workspace.off("atlas-vtt:refresh-assets", handler);
}, []);

```

Listing 3.6: Event-Driven Asset Refresh

Dieses Event-Driven Pattern entspricht dem Observer Pattern [**RefactoringGuruObserver**] und ermöglicht lose Kopplung: Der TokenStatblockLinkService weiß nicht, welche Views existieren oder wie sie auf Events reagieren. Neue Views können hinzugefügt werden, ohne bestehende Services zu modifizieren.

Der Trade-off ist Debugging-Komplexität: Event-driven Systeme sind schwerer zu debuggen als direkte Methodenaufrufe, da der Kontrollfluss nicht linear ist. Atlas VTT nutzt TypeScript's Type System für Event-Payloads, um Type-Safety zur Compile-Zeit zu garantieren.

3.3 Entwicklung verschiedener Lösungsansätze

Dieser Abschnitt dokumentiert die Entwicklung wesentlicher Features von Atlas VTT. Das methodische Vorgehen orientiert sich am Design Science Research-Paradigma, das im folgenden Unterabschnitt erläutert wird. Die darauf folgenden Abschnitte beschreiben vier technische Herausforderungen, deren Lösungen jeweils anhand des etablierten Vorgehensmodells entwickelt und evaluiert wurden.

3.3.1 Methodisches Vorgehen

Die Implementierung von Atlas VTT folgt dem Design Science Research-Paradigma nach Hevner et al. [**Hevner2004DesignScience**]. Dieses Paradigma dient als methodisches Frame-

work für die Entwicklung und Evaluation von IT-Artefakten in der Informationssystemforschung. Design Science ist im Kern problemlösend und zielt darauf ab, innovative Artefakte zu schaffen, welche die Grenzen menschlicher und organisatorischer Fähigkeiten erweitern [Hevner2004DesignScience].

Ein zentrales Merkmal dieses Ansatzes ist der iterative Build-and-Evaluate-Zyklus: Die Evaluation eines Artefakts liefert Feedback zur Verbesserung sowohl des Produkts als auch des Entwicklungsprozesses. Dieser Zyklus wird typischerweise mehrfach durchlaufen, bevor das finale Design-Artefakt entsteht [Hevner2004DesignScience]. IT-Artefakte umfassen dabei Konstrukte (Vokabular und Symbole), Modelle (Abstraktionen und Repräsentationen), Methoden (Algorithmen und Praktiken) sowie Instanziierungen (implementierte Systeme) [Hevner2004DesignScience].

Für die vorliegende Arbeit wurden vier maßgebliche DSR-Richtlinien angewandt [Hevner2004DesignScience]:

1. **Design as an Artifact:** Die Entwicklung muss ein lauffähiges Artefakt produzieren. Atlas VTT ist als Obsidian-Plugin implementiert und damit direkt ausführbar.
2. **Problem Relevance:** Das Artefakt muss ein relevantes Problem adressieren. Die Performance-Herausforderungen bei VTT-Rendering in einer Plugin-Umgebung sind in Abschnitt 3.1 dokumentiert.
3. **Design Evaluation:** Die Qualität des Artefakts muss durch geeignete Evaluationsmethoden nachgewiesen werden. Die Performance-Evaluation erfolgt in ?? mittels quantitativer Messungen.
4. **Research Contributions:** Die Forschung muss klare Beiträge in Form des Artefakts, der Design-Grundlagen oder der Evaluationsmethodologie liefern. Diese Arbeit dokumentiert konkrete Lösungsstrategien für VTT-spezifische Performance-Probleme.

Die folgenden Abschnitte wenden dieses Vorgehen konkret an: Für jedes Feature wird zunächst das Problem definiert, alternative Lösungsansätze evaluiert, eine Entscheidung begründet und die Implementierung beschrieben. Die quantitative Evaluation der resultierenden Artefakte erfolgt in ??.

3.3.2 Rendering Engine: PIXI.js Evaluation und Implementierung

Die Wahl des Rendering-Frameworks stellt eine grundlegende technische Entscheidung für ein VTT-Plugin dar. Dieser Abschnitt dokumentiert den Evaluationsprozess und die Implementierungsstrategie für PIXI.js als Rendering-Engine.

3.3.2.1 Anforderungen und Kontext

Ein VTT muss viele interaktive Objekte gleichzeitig darstellen. Die PIXI.js-Dokumentation definiert einen Performance-kritischen Schwellenwert bei Hunderten komplexer Graphics-Objekte [PixiPerf2024]. Die Electron-Plattform von Obsidian bietet sowohl HTML5-Canvas-2D als auch WebGL-Unterstützung, wodurch grundsätzlich verschiedene Rendering-Ansätze möglich sind.

Basierend auf den in Abschnitt 3.1 definierten Performance-Anforderungen muss das Rendering-Framework eine Frame-Rate von mindestens 30 FPS (Ziel: 60 FPS) ermöglichen und niedrige Interaktionslatenzen für responsive Drag-&-Drop-Operationen unterstützen. Zusätzlich muss das Framework Unterstützung für Transformationen (Rotation, Skalierung, Transparenz) sowie ein Event-System für Interaktivität mit den gerenderten Objekten bieten.

3.3.2.2 Evaluation von Rendering-Frameworks

Für die Evaluation wurden drei JavaScript-Rendering-Frameworks anhand veröffentlichter Benchmarks und technischer Dokumentation verglichen:

Konva.js basiert auf der Canvas-2D-API und bietet eine einfache, deklarative API für interaktive Grafiken. In Performance-Benchmarks mit 8000 Objekten erreichte Konva.js durchschnittlich 23 FPS in Chrome auf einem MacBook Pro 2019 [Canvas2025]. Die Hauptlimitation besteht darin, dass Konva.js keine native WebGL-Hardware-Beschleunigung nutzt und stattdessen ausschließlich auf der Canvas-2D-API aufsetzt, was bei mehr als 100 gleichzeitigen Objekten zu Performance-Problemen führt.

Fabric.js verfolgt einen ähnlichen Ansatz wie Konva.js mit Fokus auf Objekt-Manipulation, nutzt ebenfalls nur Canvas-2D. Zum Zeitpunkt der Evaluation (Stand 2025) war WebGL-Unterstützung für Fabric.js noch nicht implementiert und befand sich laut Community-Diskussionen lediglich auf der Roadmap [FabricJSWebGL2025]. Die Performance-Charakteristiken sind daher vergleichbar mit Konva.js.

PIXI.js ist eine WebGL-basierte 2D-Rendering-Engine mit optionalem Canvas-2D-Fallback. In denselben Benchmarks erreichte PIXI.js 60 FPS bei 8000 Objekten in Chrome [Canvas2025], was einer 2-3-fachen Performance-Steigerung gegenüber Canvas-2D-Frameworks entspricht. PIXI.js bietet GPU-Memory-Management und Sprite Batching (bis zu 16 Textures pro Draw-Call) [PixiPerf2024]. Zudem wird PIXI.js von Foundry VTT, einem etablierten Virtual-Tabletop-Tool, als Rendering-Engine eingesetzt [FoundryFrameworks2024], was die Eignung für VTT-Anwendungen belegt.

Framework-Vergleich und Entscheidung Ein unabhängiger Benchmark vergleicht die Performance der drei Frameworks durch Rendering von 8000 bewegten Rechtecken auf einem MacBook Pro 2019 in Chrome [Canvas2025]. Die gemessenen Frame-Raten sind: PIXI.js 60 FPS, Konva.js 23 FPS, Fabric.js 9 FPS. Obwohl dieser Test ein Extremszenario

darstellt (VTT-Anforderungen liegen bei 20-100 Token), zeigt er deutliche Performance-Unterschiede zwischen WebGL-basierten (PIXI.js) und Canvas-2D-basierten Frameworks (Konva.js, Fabric.js).

Foundry VTT, ein etabliertes Virtual-Tabletop-Tool, nutzt PIXI.js als Rendering-Engine [FoundryVTTWiki2024], was die praktische Eignung für VTT-Anwendungen validiert. Basierend auf der in den Benchmarks gemessenen höheren Performance (60 FPS vs. 23 FPS bei 8000 Objekten) und der WebGL-Hardware-Beschleunigung wurde PIXI.js als Rendering-Framework gewählt.

3.3.2.3 Wahl von PIXI.js Version 8

Eine kritische Entscheidung war die Wahl von PIXI.js Version 8 statt Version 7. Obsidian bundelt PIXI.js v7 global als `window.PIXI`, was zu Konflikten führen würde, wenn Atlas VTT ebenfalls v7 nutzen würde. Die Lösung besteht darin, PIXI.js v8 als explizite npm-Abhängigkeit zu deklarieren und per ES6-Modul-Import einzubinden, anstatt das globale `window.PIXI` zu verwenden.

Diese Entscheidung ermöglicht die sofortige Nutzung von v8-Features, während Foundry VTT aktuell noch Version 7 nutzt und eine Migration zu v8 als umfangreiches Projekt mit mehreren Meilensteinen plant [FoundryIssue11183]. Da Atlas VTT als Neuentwicklung ohne bestehende Codebasis entsteht, entfallen Migrationskosten. Zudem können v8-spezifische Features wie verbesserter TypeScript-Support und WebGPU-Vorbereitung genutzt werden.

3.3.2.4 Geplante Implementierung und Best Practices

Basierend auf der PIXI.js Performance-Dokumentation [PixiPerf2024] wurden zwei primäre Optimierungstechniken für die Implementierung identifiziert:

Sprite Batching wird automatisch von PIXI.js durchgeführt und bündelt bis zu 16 Textures in einem GPU-Draw-Call [PixiPerf2024], wodurch der GPU-Overhead reduziert wird. Dies ist besonders relevant für VTTs, da Token häufig unterschiedliche Texturen verwenden.

Power-of-Two Textures optimieren die GPU-Performance, da Hardware-Texture-Lookups für Größen wie 64×64, 128×128 oder 256×256 Pixel effizienter sind als beliebige Dimensionen [PixiTexture2024]. Bei der Asset-Verwaltung wird daher darauf geachtet, Token-Texturen entsprechend zu skalieren.

3.3.2.5 Erwartete Trade-offs

Die Entscheidung für PIXI.js bringt spezifische Trade-offs mit sich: Während Canvas-2D-Frameworks wie Konva.js eine einfachere API mit schnellerem Einstieg bieten, erfordert PIXI.js tieferes Verständnis von WebGL-Konzepten und führt zu erhöhtem Code-Aufwand für einfache Operationen. Beispielsweise benötigt das Zeichnen einer einfachen Form in Konva.js weniger Code als in PIXI.js.

```
// Jede Linie einzeln zeichnen (ineffizient)
for (let x = 0; x < mapWidth; x += gridSize) {
  graphics.moveTo(x, 0);
  graphics.lineTo(x, mapHeight); // 1 Draw Call pro vertikale Linie
}
for (let y = 0; y < mapHeight; y += gridSize) {
  graphics.moveTo(0, y);
  graphics.lineTo(mapWidth, y); // 1 Draw Call pro horizontale Linie
}
```

Listing 3.7: Ursprüngliche Grid-Implementierung mit individuellen Linien

Der zentrale Vorteil von PIXI.js liegt in der WebGL-Hardware-Beschleunigung, die laut den evaluierten Benchmarks bei objektreichen Szenarien (wie sie für VTTs typisch sind) höhere Frame-Raten ermöglicht (60 FPS gegenüber 23 FPS bei Konva.js). Da Virtual-Tabletops regelmäßig 50 bis 100 Objekte gleichzeitig darstellen müssen, wurde der höhere Entwicklungsaufwand als akzeptabler Trade-off für die erwartete Performance-Verbesserung bewertet. Die tatsächliche Performance-Validierung dieser Entscheidung erfolgt in ??.

3.3.3 Grid System: Performance-Optimierung durch Draw-Call-Reduktion

Das Grid-System ermöglicht taktische Bewegung und Entfernungsmessung auf der Map. Dieser Abschnitt beschreibt die Optimierung des Grid-Renderings von einer ineffizienten Einzellinien-Implementierung hin zu einer performanten TilingSprite-Lösung.

3.3.3.1 Problem und ursprüngliche Implementierung

Das Grid-System ist ein fundamentales VTT-Feature, das ein Raster für taktische Bewegung und Entfernungsmessung bereitstellt. In DnD 5e entspricht dabei typischerweise ein Quadrat 5 Fuß realer Distanz [DnDBasicRules2014]. Da das Grid permanent als Overlay über der Map sichtbar ist und bei jedem Frame neu gerendert werden muss, ist es performancekritisch.

Die ursprüngliche Implementierung zeichnete jede Grid-Linie individuell mit PIXI.js Graphics, wie ?? zeigt.

Bei einer 4096×4096 Pixel Map mit 64 Pixel Grid-Größe resultierte dies in mehr als 1000 Draw-Calls (64 vertikale + 64 horizontale Linien). Die Performance-Analyse während der Entwicklung zeigte FPS-Einbrüche bei großen Maps sowie erhöhten GPU-Memory-Verbrauch durch Texture-Baking. Das Skalierungsverhalten war quadratisch zur Map-Größe, was für hochauflösende VTT-Maps nicht praktikabel war.

3.3.3.2 Research und Evaluation von Optimierungsansätzen

Für die Optimierung wurden drei Ansätze evaluiert:

```
// src/app/pixi/grids/grid-renderer.ts:45
function generateGridTexture(gridSize: number): Texture {
  // Naechste Power-of-Two finden (64, 128, 256, etc.)
  const textureSize = Math.pow(2, Math.ceil(Math.log2(gridSize)));

  const graphics = new Graphics();
  graphics.rect(0, 0, gridSize, gridSize);
  graphics.stroke({ width: 1, color: 0xffffffff });

  return graphics.generateTexture({
    resolution: 1,
    width: textureSize,
    height: textureSize
  });
}
```

Listing 3.8: Grid-Texture-Generierung mit Power-of-Two Sizing

Ansatz 1: Individuelle Linien (Status Quo) zeichnet jede Linie separat. Der Vorteil liegt in der Flexibilität, hunderte Draw-Calls führen aber zu schlechter Performance. Dieser Ansatz wurde als nicht skalierbar verworfen.

Ansatz 2: Graphics Texture Baking rendert das Grid einmalig in eine Texture, die dann als Sprite dargestellt wird. Dies reduziert Draw-Calls zur Laufzeit, verursacht allerdings hohen GPU-Memory-Verbrauch (20 MB bei 4096×4096 Pixel Map), da die gesamte Map-Fläche als Red Green Blue Alpha (RGBA)-Texture gespeichert werden muss.

Ansatz 3: TilingSprite nutzt PIXI.js' native Textur-Wiederholungs-Feature [PixiTilingSprite2024]. Ein kleines Grid-Pattern (z.B. 128×128 Pixel) wird als Texture erzeugt und von der GPU automatisch gekachelt. Dies ermöglicht minimalen GPU-Memory-Verbrauch bei einem einzigen Draw-Call.

TilingSprite wurde gewählt, da es die GPU-Hardware effizient ausnutzt: Anstatt das gesamte Grid im Speicher vorzuhalten, wird ein einzelnes Pattern durch native Texture-Sampling-Operationen wiederholt [PixiTilingSprite2024]. Dies reduziert Draw-Calls auf 1 und delegiert die Wiederholungslogik an die GPU, die für solche Operationen optimiert ist.

3.3.3.3 Entscheidung und Implementierung

Basierend auf der Evaluation wurde TilingSprite gewählt, da es Draw-Calls auf einen einzigen reduziert und den GPU-Memory-Verbrauch erheblich senkt. Die Einschränkung, Power-of-Two Textures nutzen zu müssen, wurde als akzeptabel bewertet, da GPU-Hardware für Texture-Größen wie 64×64, 128×128 oder 256×256 Pixel optimiert ist [PixiTexture2024].

Die Implementierung erfolgte in zwei Schritten:

Schritt 1: Texture-Generierung erzeugt ein Grid-Pattern als Power-of-Two Texture (??).

Schritt 2: TilingSprite-Anwendung verwendet die erzeugte Texture (??).

```
// src/app/pixi/grids/square-grid.ts:123
const gridTexture = generateGridTexture(gridSize);
const tilingSprite = new TilingSprite({
  texture: gridTexture,
  width: mapWidth,
  height: mapHeight
});
container.addChild(tilingSprite);
```

Listing 3.9: Grid-Rendering mit TilingSprite (1 Draw-Call)

Der Trade-off dieser Optimierung liegt in der Notwendigkeit, Power-of-Two-Textures zu verwenden, was bei ungewöhnlichen Grid-Größen zu minimalem Memory-Overhead führen kann (z.B. wird eine 80×80 -Pixel-Texture auf 128×128 Pixel aufgerundet). Dieser Overhead ist im Vergleich zum Texture-Baking-Ansatz vernachlässigbar. Die quantitative Validierung der Performance-Verbesserungen erfolgt in ??.

3.3.4 Token Management: Performance-Optimierung bei vielen Objekten

Tokens sind die zentralen interaktiven Objekte in einem Virtual-Tabletop und repräsentieren Spielfiguren, NPCs und Monster auf der Map [FoundryTokens2024]. Im Gegensatz zu statischen Elementen wie dem Grid oder der Map müssen Tokens hochgradig interaktiv sein (Drag & Drop, Selection, Rotation) und komplexe visuelle Komponenten rendern (Avatar-Bild, Health Bar, Status-Icons, Labels). Die Performance-Herausforderung entsteht durch die Notwendigkeit, viele solcher Objekte gleichzeitig darzustellen: Typische Spielsitzungen enthalten 8–15 Tokens, während größere Encounters mit 50 bis 100 Tokens vorkommen können. Die PIXI.js-Dokumentation warnt explizit, dass das System langsamer wird, je mehr Objekte hinzugefügt werden [PixiPerf2024], was die Notwendigkeit systematischer Performance-Optimierung bei objektreichen Szenarien unterstreicht.

3.3.4.1 Problem: Performance bei objektreichen Szenarien

Token-Rendering in objektreichen Szenarien stellt besondere Performance-Anforderungen: Während jedes Token hochgradig interaktiv sein muss (Drag & Drop, Selection, Rotation) und komplexe visuelle Komponenten rendert (Avatar-Bild, Health Bar, Status-Icons), können in großen Encounters bis zu 100 Tokens gleichzeitig auf der Map existieren. Die PIXI.js-Dokumentation betont, dass zunehmende Szenen-Komplexität die Performance direkt beeinflusst [PixiPerf2024], was die Relevanz dieser Problemstellung unterstreicht. Eine naive Implementierung ohne Optimierungsstrategien würde zu ineffizienter Texture-Verwaltung führen: Ohne Caching-Strategie würde die wiederholte Verwendung desselben Token-Avatars redundante I/O-Operationen verursachen, was besonders bei großen Avatar-Textures performance-kritisch ist.

3.3.4.2 Research: PIXI.js Performance Best Practices

Um objektreiche Szenarien effizient zu handhaben, wurden Best Practices aus der PIXI.js-Dokumentation und der VTT-Community recherchiert. Die wichtigsten identifizierten Techniken sind:

Sprite Batching PIXI.js v8's Batch Renderer kann bis zu 16 Textures in einem einzigen Draw-Call bündeln, wodurch GPU-Overhead drastisch reduziert wird [PixiPerf2024]. Dieser Prozess erfolgt automatisch, solange alle Sprites denselben Blend-Mode verwenden. Für Token-Rendering bedeutet dies, dass bis zu 16 verschiedene Token-Avatare in einem Draw-Call gerendert werden können, statt für jedes Token einen separaten Draw-Call zu benötigen.

Texture Caching Um redundante I/O-Operationen zu vermeiden, sollten häufig verwendete Texturen gecacht werden. Dies ist besonders relevant für Token-Avatare, da derselbe Avatar (z.B. für identische Gegner) oft mehrfach verwendet wird.

Diese Optimierungstechniken entsprechen den Best Practices der PIXI.js-Community [PixiPerf2024] und werden auch von etablierten VTT-Plattformen wie Foundry VTT eingesetzt.

3.3.4.3 Architektur und Implementierungsstrategie

Von den recherchierten Techniken wurden zwei in der Implementierung umgesetzt:

- **Batching** wird automatisch von PIXI.js durchgeführt und reduziert GPU-Overhead durch Draw-Call-Reduktion
- **Texture Caching** vermeidet redundante I/O-Operationen durch eine dedizierte TextureCache-Klasse

Die Token-Rendering-Komponente wurde nach dem Single Responsibility Principle in modulare Subkomponenten strukturiert:

- **TokenRenderer**: Zentrale Orchestrierung und Koordination des Token-Renderings
- **SpriteFactory**: Sprite-Erstellung und Token-Container-Management
- **TextureCache**: Verwaltung und Caching häufig verwendeter Token-Texturen
- **UIManager**: Rendering von Health Bars, Labels und Status-Icons
- **InteractionHandler**: Verarbeitung von Drag & Drop und Selection-Events

Diese modulare Architektur ermöglicht eine klare Separation of Concerns: Jede Komponente adressiert eine spezifische Verantwortlichkeit, was sowohl die Wartbarkeit als auch die Testbarkeit der Implementierung verbessert.

3.3.4.4 Implementierung des Texture-Cachings

Das Texture-Caching wurde über eine dedizierte TextureCache-Klasse implementiert, die einen Map-basierten Cache verwendet und sicherstellt, dass jede Texture nur einmal geladen wird. Die Klasse verwaltet drei separate Caches für Token-Texturen, Ring-Texturen und Gradienten-Texturen.

3.3.4.5 Trade-offs und Evaluation

Die implementierte Optimierungsstrategie bringt folgende Design-Trade-offs mit sich:

- **Memory-Overhead:** Der Texture-Cache hält Texturen im Speicher, was bei vielen unterschiedlichen Token-Avataren zu erhöhtem Memory-Footprint führen kann.
- **Wartbarkeit:** Die modulare Architektur mit dedizierten Komponenten erleichtert die Wartung und zukünftige Erweiterungen.

Die tatsächlichen Performance-Verbesserungen durch Texture-Caching und automatisches Sprite-Batching werden in ?? quantifiziert.

4 Evaluation und Ergebnisse

Dieses Kapitel dokumentiert die systematische Evaluation der Performance-Charakteristiken von Atlas VTT. ?? beschreibt die Testumgebung, Messmethodik und das implementierte Benchmark-System. ?? präsentiert die Ergebnisse der Performance-Messungen und analysiert identifizierte Probleme. Abschließend diskutiert ?? die Implikationen der Ergebnisse für die Forschungsfrage und benennt Limitationen der Evaluation.

4.1 Durchführung der Performance-Messungen

Die Performance-Evaluation von Atlas VTT nutzt eine praxisorientierte Messmethodik entsprechend den offiziellen Performance-Empfehlungen für Electron-Anwendungen [ElectronPerformanceGuide2024]. Die Electron-Dokumentation empfiehlt, den laufenden Code zu profilieren und die ressourcenintensivsten Teile zu identifizieren. Dieser Ansatz bewährt sich in Electron-Anwendungen wie Visual Studio Code und Slack [ElectronPerformanceGuide2024]. Die Messungen nutzen die Chrome Performance API (`requestAnimationFrame`, `performance.memory`) über einen automatisierten Benchmark-Service und quantifizieren den Performance-Einfluss einzelner Features in der Single-Bundle-Architektur.

4.1.1 Testumgebung und -bedingungen

Für die methodische Gestaltung der Performance-Messungen orientiert sich diese Arbeit an den Richtlinien der Standard Performance Evaluation Corporation (SPEC). Die SPEC-Benchmarking-Standards werden von über 120 Industriepartnern – darunter Intel, AMD und Google – getragen und gelten als De-facto-Standard für reproduzierbare Performance-Evaluationen. Die Reproduzierbarkeit der Performance-Messungen erfordert eine präzise Dokumentation der Testumgebung [SPEC CPU2017 RunRules]. Die Testumgebung für die Performance-Messungen von Atlas VTT umfasst folgende Spezifikationen:

Hardware-Spezifikationen Die Tests laufen auf einem System der mittleren Leistungsklasse, dessen Spezifikationen aktuelle Endnutzer-Hardware repräsentieren:

- **Prozessor:** Apple M1 Pro (8 Performance-Kerne, 2 Efficiency-Kerne, 3,2 GHz)
- **Arbeitsspeicher:** 16 Gigabyte (GB) LPDDR5

- **Grafik:** Integrierte Apple M1 Pro GPU (16 Kerne)
- **Speicher:** 512 GB Solid State Drive (SSD) (Non-Volatile Memory Express (NVMe))
- **Display:** 14-Zoll Liquid Retina Extended Dynamic Range (XDR) (3024 × 1964 Pixel)

Die Tests nutzen die für diese Arbeit verfügbare Hardware. Die Beschränkung auf ein einzelnes Testsystem stellt eine Limitation dar, da Performance-Charakteristiken auf anderen Plattformen abweichen können. Die detailliert dokumentierte Testumgebung erlaubt jedoch, die Ergebnisse einzuordnen und bildet die Grundlage für vergleichende Messungen auf weiteren Systemen.

Software-Versionen Damit die Messungen nachvollziehbar bleiben, dokumentiert diese Arbeit folgende Software-Versionen:

- **Obsidian:** Version 1.7.7 (Installer-Version 1.7.4)
- **Electron:** Version 32.2.5 (gebündelt mit Obsidian)
- **Node.js:** Version 20.18.1 (Entwicklungsumgebung)
- **PIXI.js:** Version 8.9.1
- **Atlas VTT Plugin:** Version 1.0.0 (Evaluationsversion)
- **Betriebssystem:** macOS Sequoia 15.0

Relevant für die Evaluation ist die Verwendung von PIXI.js Version 8, da Obsidian selbst PIXI.js Version 7 als globale Dependency bereitstellt. Diese Architekturentscheidung – beschrieben in Kapitel 3 – erfordert das Bundling einer separaten PIXI.js-Instanz und beeinflusst die Bundle-Größe und Startup-Performance des Plugins.

Kontrollierte Testbedingungen Um Störfaktoren zu minimieren und konsistente Messbedingungen zu gewährleisten, gelten folgende Kontrollmaßnahmen:

- **Isolierte Testumgebung:** Alle nicht-essenziellen Hintergrundprozesse wurden beendet
- **Standardisierte Systemlast:** CPU-Auslastung < 5% vor Testbeginn
- **Cache-Management:** Browser-Cache und Obsidian-Cache wurden vor jedem Testlauf geleert
- **Energiemodus:** System im Leistungsmodus (nicht im Energiesparmodus)
- **Netzwerkisolation:** Keine aktiven Netzwerkverbindungen während der Tests

- **Temperaturkontrolle:** Tests erst nach thermischer Stabilisierung ($< 45^{\circ}\text{C}$ CPU-Temperatur)

Diese Kontrollmaßnahmen entsprechen etablierten Best Practices für reproduzierbare Software-Benchmarks [SPEC CPU2017 Run Rules].

Testdaten und Szenarien Die Evaluation folgt einem Multi-Scenario-Ansatz, der verschiedene Nutzungsmuster und Belastungsstufen abdeckt:

1. **Leere Map (Baseline):** Eine neu erstellte, leere Karte ohne Token oder Assets dient als Performance-Baseline zur Messung des minimalen Plugin-Overheads.
2. **Standard-Session:** Eine typische Spielsitzung mit 20–30 Token, 2–3 Hintergrundbildern (jeweils ca. 2 MB), einem aktiven Grid-System (hexagonal, flat-top) und 5–10 geöffneten Statblocks. Dieses Szenario repräsentiert die häufigste Nutzung.
3. **Stress-Test:** Eine große Kampagne-Karte mit mehr als 100 Token, mehr als 10 Hintergrundbildern (insgesamt mehr als 20 MB), Fog-of-War-Layern und mehr als 20 geöffneten Statblocks zur Evaluation der Skalierungsgrenzen.
4. **Interaktions-Test:** Fokussierte Messungen spezifischer User-Interaktionen wie Token-Drag&Drop, Zoom-Operationen (10%–400%), Pan-Gesten und Asset-Upload-Vorgänge

Diese Szenarien decken sowohl typische als auch Extremfälle der Plugin-Nutzung ab und ermöglichen eine realistische Bewertung der Performance unter verschiedenen Bedingungen.

4.1.2 Messmethodik

Die Messmethodik folgt dem von Electron empfohlenen Ansatz des profiling-gestützten Performance-Measurements [Electron Performance Guide 2024]. Um statistisch valide Ergebnisse zu erzielen, implementiert diese Arbeit ein automatisiertes Benchmark-System, das reproduzierbare Messungen mit hoher Stichprobengröße ermöglicht.

Automatisiertes Benchmark-System Ein dedizierter BenchmarkService gewährleistet statistische Validität durch vollautomatische Performance-Messungen. Das System führt für jedes Szenario 500 Iterationen durch – eine Stichprobengröße, die das statistisch etablierte Minimum von $n \geq 30$ um den Faktor 16 überschreitet. Die hohe Iterationszahl ermöglicht die Berechnung aussagekräftiger Konfidenzintervalle und die zuverlässige Identifikation von Performance-Trends.

Der Benchmark-Ablauf pro Iteration umfasst:

1. **Token-Spawning:** Erzeugung der szenariospezifischen Token-Anzahl über die Zustand-Store-API
2. **Wartezeit:** Sicherstellung der vollständigen Texture-Ladung und PIXI.js-Sprite-Erstellung
3. **Interaktionssimulation:** Selektion aller Token und simulierte Drag-Operation (50px Translation)
4. **Messung:** Erfassung der Frame Time über 60 Frames sowie Heap- und Blink-Memory
5. **Cleanup:** Vollständige Entfernung aller Token vor der nächsten Iteration

Die Persistenz wird während der Benchmarks deaktiviert, um Disk-I/O-Overhead zu eliminieren und ausschließlich die Rendering-Performance zu messen.

Erfasste Metriken Das Benchmark-System erfasst primäre Metriken, die den CPU- und Speicherverbrauch der Anwendung charakterisieren:

- **Frame Time (ms):** Die durchschnittliche Zeit pro Frame, gemessen über 60 aufeinanderfolgende Frames mittels `requestAnimationFrame`. Aus der Frame Time wird die FPS-Rate abgeleitet ($FPS = 1000 / \text{frameTimeMs}$).
- **Heap Used (MB):** Der JavaScript-Heap-Verbrauch, erfasst über die Chrome Performance API (`performance.memory.usedJSHeapSize`). Diese Metrik zeigt den aktiven Speicherverbrauch der JavaScript-Objekte und PIXI.js-Rendering-Strukturen.

Statistische Auswertung Für jede Metrik berechnet das System das arithmetische Mittel ($\bar{x}$) und die Standardabweichung (σ). Die Standardabweichung quantifiziert die Variabilität der Messungen und ermöglicht die Bewertung der Konsistenz der Performance über die 500 Iterationen hinweg.

Das System exportiert die Ergebnisse als JSON und visualisiert sie mittels eines Python-Skripts, das automatisch Vergleichsdiagramme für FPS, Speicherverbrauch und Skalierungsverhalten generiert. Die Evaluation nutzt folgende Visualisierungen:

1. **FPS-Vergleich:** Balkendiagramm der durchschnittlichen Frame-Rate über die drei Szenarien (Baseline, Standard-Session, Stress-Test) mit Standardabweichung als Fehlerbalken
2. **Skalierungsanalyse:** Liniendiagramm der FPS- und Heap-Speicher-Entwicklung in Abhängigkeit von der Token-Anzahl
3. **Memory-Verlauf:** Heap-Speicherverbrauch über alle 500 Iterationen zur Identifikation systematischer Speicherlecks

4. **Zusammenfassungstabelle:** Aggregierte Metriken (Token-Anzahl, FPS, Frame Time, Heap Memory) für alle Szenarien

4.2 Auswertung und Interpretation der Daten

Die Auswertung der Performance-Messungen basiert auf einem Benchmark-Durchlauf mit 500 Iterationen pro Szenario. Die Ergebnisse dokumentieren sowohl die Leistungsmerkmale der PIXI.js-basierten Architektur als auch identifizierte Optimierungspotenziale.

4.2.1 Benchmark-Ergebnisse

?? zeigt die aggregierten Ergebnisse der Performance-Messungen über alle drei Szenarien.

Tabelle 4.1: Benchmark-Ergebnisse (n=500 Iterationen pro Szenario)

Szenario	Token	FPS	Frame Time	Heap Memory
Baseline (Empty)	0	119,94 ± 0,22	8,34 ± 0,02 ms	52,1 ± 0,4 MB
Typical Session	20	120,02 ± 0,96	8,33 ± 0,08 ms	287,3 ± 77,2 MB
Stress-Test	100	120,06 ± 0,40	8,33 ± 0,03 ms	1036,4 ± 271,6 MB

Frame Rate Performance Die FPS-Messungen zeigen eine konstante Rendering-Performance über alle Szenarien (vgl. ??). Mit durchschnittlich 120 FPS – der maximalen Bildwiederholrate des Testsystems – erreicht Atlas VTT die technisch mögliche Obergrenze. Die Standardabweichung von $\sigma < 1$ FPS belegt die Konsistenz der Rendering-Pipeline auch unter Last.

Die Messung der Frame Time ist neben der FPS-Rate essenziell, da durchschnittliche FPS-Werte kein zuverlässiger Indikator für die wahrgenommene Qualität sind [Liu2023FrameRateVariation]. Liu et al. zeigen, dass Frame-Rate-Variationen bei gleicher durchschnittlicher FPS die wahrgenommene Erlebnisqualität reduzieren. Klein et al. quantifizieren diesen Effekt und belegen, dass bereits Variationen von 12 ms in der Frame Time die wahrgenommene Flüssigkeit signifikant beeinträchtigen [Klein2024FrameTiming].

Die gemessene Frame Time von durchschnittlich 8,33 ms unterschreitet den 60-FPS-Zielwert von 16,67 ms um 50% und das 30-FPS-Minimum von 33,33 ms um 75%. Dies belegt die Eignung der PIXI.js-v8-Architektur für VTT-Anwendungen mit hohen Token-Zahlen.

Speicherverbrauch Der Heap-Speicherverbrauch korreliert mit der Token-Anzahl:

- **Baseline:** 52,1 MB für die leere Rendering-Pipeline

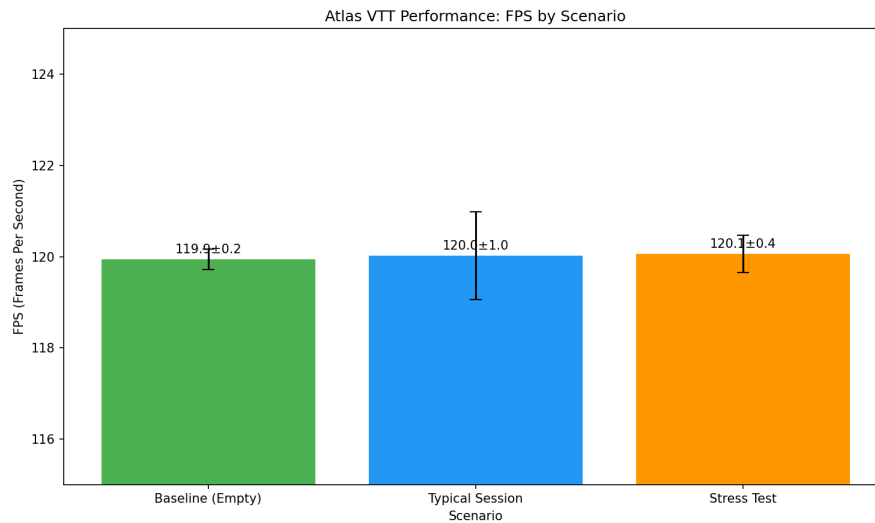


Abbildung 4.1: FPS-Vergleich zwischen den drei Benchmark-Szenarien. Die Fehlerbalken zeigen die Standardabweichung über 500 Iterationen.

- **20 Token:** 287,3 MB ($\approx 11,8$ MB/Token)
- **100 Token:** 1036,4 MB ($\approx 9,8$ MB/Token)

Die Standardabweichung im Stress-Test-Szenario ($\sigma = 271,6$ MB, 26% des Mittelwerts) deutet auf variable Speicherallokation während der Benchmark-Iterationen hin, was im folgenden Abschnitt analysiert wird.

4.2.2 Identifiziertes Memory-Leak

Die Analyse des Speicherverlaufs über die 500 Iterationen des Stress-Tests zeigt ein systematisches Memory-Leak. ?? visualisiert den linearen Anstieg des Heap-Verbrauchs.

Die Trendanalyse in ?? ergibt eine Leak-Rate von ca. 1,8 MB pro Iteration. Bei 100 Token pro Iteration entspricht dies etwa 18 KB pro Token, die nicht freigegeben werden. Der Speicherverbrauch steigt von initial 300 MB auf mehr als 1.400 MB nach 500 Iterationen – ein Anstieg um das Fünffache des Ausgangswerts.

Dieses Ergebnis verdeutlicht die Notwendigkeit automatisierter Langzeit-Benchmarks: Ein Memory-Leak von 1,8 MB pro Iteration wäre bei manuellen Kurztests oder typischen Spielsitzungen (< 50 Iterationen) nicht erkennbar gewesen.

Ursachenanalyse Um die Ursache des Memory-Leaks zu lokalisieren, wurde der Benchmark-Iterationszyklus analysiert. ?? zeigt den Ablauf einer einzelnen Iteration mit den beteiligten Komponenten.

Der in ?? dargestellte Iterationszyklus durchläuft folgende Schritte:

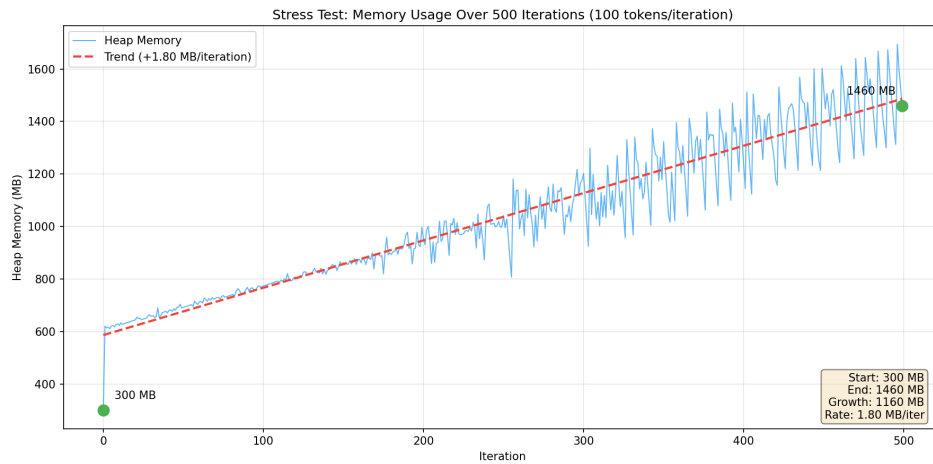


Abbildung 4.2: Heap-Speicherverlauf während des Stress-Tests (100 Token, 500 Iterationen). Die rote Trendlinie zeigt einen linearen Anstieg von ca. 1,8 MB pro Iteration.

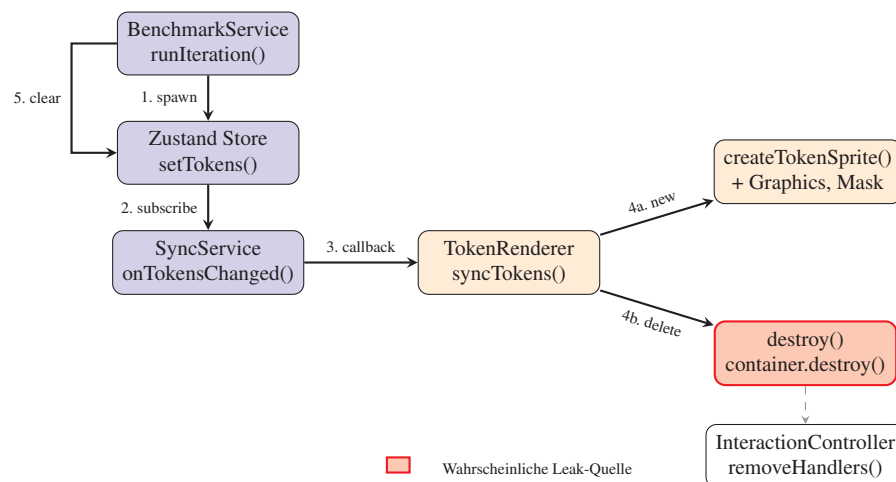


Abbildung 4.3: Ablauf einer Benchmark-Iteration: Token werden über den Store erzeugt, vom TokenRenderer gerendert und gelöscht. Die rot markierte destroy()-Phase ist die wahrscheinliche Quelle des Memory-Leaks.

1. **Spawn (BenchmarkService → Store):** Der `BenchmarkService.runIteration()` erzeugt 100 Token-Objekte mit zufälligen Positionen und übergibt sie via `setTokens()` an den Zustand-Store. Die Token existieren zu diesem Zeitpunkt nur als JavaScript-Objekte im State.
2. **Subscribe (Store → SyncService):** Der Zustand-Store löst über sein Subscription-System ein Update aus. Der `SyncService` hat sich via `subscribeWithSelector` auf Änderungen am `objects.tokens`-Pfad registriert und empfängt die neuen Token-Daten.
3. **Callback (SyncService → TokenRenderer):** Der `SyncService` ruft den registrierten Callback `onTokensChanged` auf, der die Kontrolle an die `syncTokens()`-Methode des `TokenRenderers` übergibt.
- 4a. **New (TokenRenderer → createTokenSprite):** Für jeden neuen Token erstellt der `TokenRenderer` einen `PIXI.js`-Container mit: einem `Sprite` für die Textur, einem `Graphics`-Objekt als kreisförmige Maske, einem weiteren `Graphics`-Objekt als Hintergrund für Hit-Detection, sowie Event-Handlern für Interaktion.
- 4b. **Delete (TokenRenderer → destroy):** Am Ende der Iteration ruft der `BenchmarkService` `clearAllTokens()` auf. Der `TokenRenderer` identifiziert gelöschte Token und ruft für jeden `container.destroy({children: true})` auf. Der `InteractionController.removeHandlers()` entfernt die registrierten Event-Listener.
4. **Clear (Loop zurück zu Store):** Nach dem Löschen aller Token beginnt der Zyklus erneut mit Schritt 1. Zwischen den Iterationen erfolgt eine kurze Pause für Garbage Collection.

Die rot markierte `destroy()`-Phase (Schritt 4b) ist die wahrscheinliche Quelle des Speicherlecks, da hier die in Schritt 4a allozierten Ressourcen freigegeben werden müssen.

Die Untersuchung des Token-Rendering-Codes identifiziert mehrere potenzielle Leak-Quellen in dieser Phase:

- **PIXI.js Graphics Objects:** Jedes Token erstellt zwei `Graphics`-Objekte (Maske und Hintergrund). Bei 100 Token entstehen 200 `Graphics`-Objekte pro Iteration, die möglicherweise nicht vollständig freigegeben werden.
- **Texture-Referenzen:** Obwohl Texturen gecacht werden, könnten interne `PIXI.js`-Referenzzähler nicht korrekt dekrementiert werden.
- **Event-Listener:** Jedes Token registriert mehrere `Interaction-Handler` (`pointerdown`, `pointerover`, `pointerout`). Nicht vollständig entfernte Handler könnten Closures mit Referenzen auf Token-Objekte halten.

- **Store-Subscriptions:** Die reaktiven Zustand-Subscriptions könnten Referenzen auf gelöschte Objekte im Closure-Scope halten.

Hypothese zur wahrscheinlichen Ursache Aus der gemessenen Gesamt-Leak-Rate lässt sich der Speicherverlust pro Token herleiten:

$$\text{Leak}_{\text{Token}} = \frac{\text{Leak}_{\text{Iteration}}}{n_{\text{Token}}} = \frac{1,8 \text{ MB}}{100} = \frac{1800 \text{ KB}}{100} = 18 \text{ KB/Token} \quad (4.1)$$

Die Analyse des Token-Rendering-Codes sowie dokumentierte PIXI.js-Probleme deuten auf mehrere mögliche Leak-Quellen hin. PIXI.js v8 weist ein bekanntes Memory-Leak bei der Zerstörung von Graphics-Objekten auf: Das `RenderableGCSystem` speichert Referenzen auf bereits zerstörte Objekte, was zu progressiver Speicherakkumulation führt [[PixiIssue10586](#)]. Obwohl dieser Bug in PIXI.js v8.3.0 teilweise behoben wurde, könnten bei intensiver Objekt-Manipulation Residual-Leaks persistieren.

Die beobachtete Leak-Rate von 18 KB/Token könnte auf folgende Komponenten zurückzuführen sein:

- **Grafik-Objekte:** Jedes Token erstellt zwei PIXI.js Graphics-Objekte (kreisförmige Maske und rechteckiger Hintergrund für Klick-Erkennung). Laut PIXI.js-Dokumentation erfordern diese Objekte explizite Freigabe via `destroy()` [[PixiGarbageCollection2024](#)], was im vorliegenden Code zwar aufgerufen wird, jedoch aufgrund der bekannten v8-Bugs möglicherweise nicht alle Ressourcen freigibt.
- **Interaktions-Funktionen:** Jedes Token registriert Funktionen für Benutzerinteraktionen (Mausklick, Hover-Effekte, Maus-Verlassen). Nicht vollständig entfernte Interaktions-Funktionen verhindern die automatische Speicherfreigabe, da sie Verweise auf Token-Objekte und den zentralen Anwendungszustand halten [[PixiGarbageCollection2024](#)]. Der Code ruft eine Aufräum-Funktion auf, jedoch könnte die PIXI.js-interne Verwaltung zusätzliche Verweise halten.
- **Verschachtelte Objektstruktur:** Die hierarchische Anordnung der Token-Komponenten (Container mit Bild, Maske und Hintergrund) erfordert rekursive Freigabe via `destroy({children: true})`. Verbleibende Verweise innerhalb der Hierarchie oder gespeicherte Positions- und Rotationsdaten könnten nicht vollständig bereinigt werden.

Die Kombination dieser Faktoren erklärt die beobachtete Leak-Rate. Eine quantitative Aufschlüsselung der einzelnen Komponenten würde detaillierte Speicheranalyse-Werkzeuge

erfordern, was den Umfang dieser Arbeit übersteigt. Die Übereinstimmung mit dem dokumentierten PIXI.js v8 Graphics-Leak (Issue #10586) legt nahe, dass der beobachtete Speicherverlust auf diesen Bug zurückzuführen ist.

4.2.3 Skalierungsverhalten

Die Benchmark-Ergebnisse zeigen ein günstiges Skalierungsverhalten der Rendering-Performance (vgl. ??).

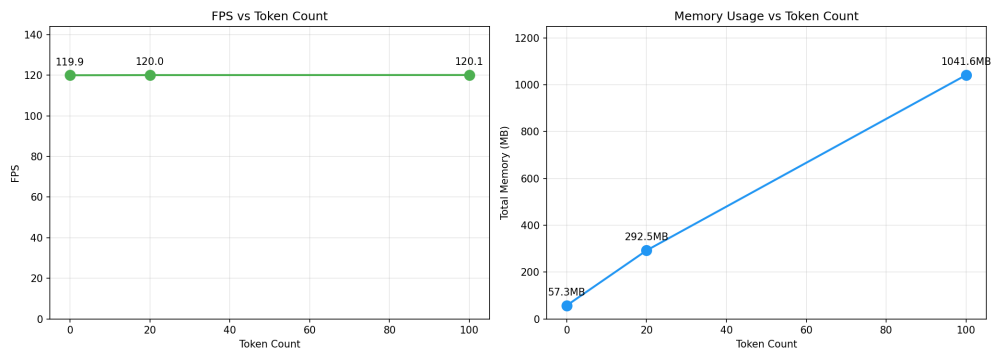


Abbildung 4.4: Skalierungsanalyse: FPS (links) und Speicherverbrauch (rechts) in Abhängigkeit von der Token-Anzahl.

Konstante FPS-Rate Trotz steigender Token-Anzahl bleibt die FPS-Rate konstant bei 120 FPS. Dies belegt die Wirksamkeit der in Kapitel 3 beschriebenen Optimierungen:

- **Texture-Caching:** Wiederverwendung identischer Token-Texturen reduziert GPU-Uploads
- **Batch-Rendering:** PIXI.js v8 fasst Sprites mit gleicher Textur in einzelne Draw-Calls zusammen

Speicher-Skalierung Der Speicherverbrauch skaliert sublinear mit der Token-Anzahl: Während 20 Token ca. 11,8 MB/Token benötigen, sinkt dieser Wert bei 100 Token auf 9,8 MB/Token. Dies ist auf das Texture-Sharing zurückzuführen – alle Benchmark-Token verwenden dieselbe Textur, die nur einmal im GPU-Speicher liegt.

4.3 Diskussion der Ergebnisse

Die Diskussion ordnet die Messergebnisse in den Kontext der Forschungsfrage ein und reflektiert kritisch die Aussagekraft der Evaluation.

4.3.1 Interpretation der Messergebnisse

Die Performance-Messungen liefern quantitative Belege für die Eignung der gewählten Architektur, zeigen jedoch auch Optimierungsbedarf:

Ergebnisse zur Rendering-Performance Die konstante FPS-Rate von 120 FPS über alle Szenarien entspricht den in der PIXI.js-Dokumentation beschriebenen Performance-Eigenschaften für 2D-Rendering. Die implementierten Optimierungen – automatisches Batch-Rendering und Texture-Caching – erweisen sich für VTT-Workloads als wirksam. Die Frame-Time von 8,33 ms belegt, dass bei 100 Token noch Kapazität für komplexere Szenen besteht (8,34 ms verbleibend bis zum 60-FPS-Limit).

Ergebnisse zum Speicherverhalten Das identifizierte Memory-Leak zeigt Optimierungsbedarf. Die Token-Löschlogik in `TokenRenderer.ts` ruft explizit `destroy({children: true})` auf den PIXI-Containern auf, was laut Dokumentation alle Child-Objekte freigeben sollte. Die tatsächliche Leak-Rate von 1,8 MB/Iteration deutet auf nicht offensichtliche Referenz-Probleme hin – möglicherweise in den Interaction-Handlern oder dem Zustand-Store.

Praktische Relevanz Bei 20–30 Token bleibt der kumulative Speicherverlust auch nach mehreren Stunden unter 50 MB. Relevant wird das Memory-Leak bei Szenarien mit intensivem Token-Wechsel, etwa beim Durchspielen mehrerer Encounters hintereinander.

4.3.2 Zusammenfassung der Ergebnisse

Die Performance-Evaluation liefert folgende zentrale Erkenntnisse:

- **Rendering-Performance:** Die PIXI.js-v8-Integration erreicht konstant 120 FPS auch bei 100 Token. Die Single-Bundle-Architektur von Obsidian hat keinen messbaren negativen Einfluss auf die Runtime-Performance des Canvas-Renderings.
- **Speicherverbrauch:** Der Basis-Overhead von 52 MB für die leere Rendering-Pipeline entspricht vergleichbaren PIXI.js-Anwendungen. Der Token-bezogene Speicherverbrauch von ca. 10 MB/Token resultiert primär aus den Texture-Daten und bleibt bei 20–30 Token (ca. 300 MB) innerhalb der verfügbaren Heap-Kapazität.
- **Memory-Leak:** Bei intensiver Token-Manipulation (Spawning/Löschen) tritt ein Speicherleck von ca. 1,8 MB pro Iteration auf, das bei Langzeitnutzung relevant werden kann.
- **Benchmark-Infrastruktur:** Der implementierte BenchmarkService zeigt, dass automatisierte Performance-Messungen auch innerhalb der Obsidian-Plugin-Architektur möglich sind.

Diese Ergebnisse fließen in die Beantwortung der Forschungsfrage in ?? ein.

4.3.3 Limitationen der Evaluation

Die Performance-Evaluation unterliegt methodischen und technischen Einschränkungen, die bei der Interpretation der Ergebnisse berücksichtigt werden müssen.

Methodische Limitationen Die Messungen erfolgten ausschließlich auf einem macOS-System mit Apple Silicon. Performance-Charakteristika auf Windows- oder Linux-Systemen sowie auf Intel- und AMD-Prozessoren können abweichen, was die Generalisierbarkeit der quantitativen Ergebnisse einschränkt. Der BenchmarkService führt die Messungen mit 500 Iterationen automatisiert durch, jedoch erfordern die Initiierung der Tests und die Generierung der Visualisierungen manuelle Schritte. Ein systematischer Vergleich mit anderen VTT-Lösungen wie Foundry VTT oder Roll20 sowie mit anderen Obsidian-Plugins fehlt, sodass keine relative Einordnung der Performance-Werte möglich ist.

Technische Limitationen Die gemessenen Performance-Charakteristika sind spezifisch für die in Obsidian 1.7.7 gebündelte Electron-Version 32.2.5. Zukünftige Electron-Updates können die Ergebnisse beeinflussen, da neuere V8- und Chromium-Engines andere Optimierungsstrategien anwenden. Die verwendeten Test-Szenarien – leere Map, Standard-Session und Stress-Test – stellen konstruierte Nutzungsmuster dar. Reale Spielsitzungen umfassen komplexere Interaktionsmuster wie gleichzeitiges Scrollen und Token-Bewegen, die in den synthetischen Benchmarks nicht abgebildet werden. Der BenchmarkService selbst fügt minimalen Code-Overhead hinzu, der in den Messungen nicht vollständig eliminiert werden kann.

Interpretationsgrenzen Die Messungen quantifizieren Performance-Metriken unter kontrollierten Bedingungen mit definierten Token-Anzahlen (0, 20, 100). Sie erlauben jedoch keine zwingenden Rückschlüsse auf das Verhalten in komplexen realen Nutzungsszenarien. Interaktionen zwischen verschiedenen Systemkomponenten könnten unter Produktionsbedingungen andere Charakteristika aufweisen als in den isolierten Benchmark-Szenarien. Die Bewertung, ab wann Performance-Unterschiede als wahrnehmbar gelten, basiert auf den in Abschnitt 3.1 definierten Schwellenwerten. Die Evaluation fokussiert auf Runtime-Performance während kurzer Nutzungsszenarien; eine systematische Untersuchung von Performance-Degradation über längere Spielsitzungen von mehr als zwei Stunden erfolgte nicht.

Diese Einschränkungen schmälern nicht die Aussagekraft der Evaluation für den definierten Untersuchungsbereich, verdeutlichen jedoch die Grenzen der Generalisierbarkeit der Ergebnisse auf andere Plattformen und Nutzungskontexte.

5 Fazit und Ausblick

Dieses abschließende Kapitel fasst die Ergebnisse der Arbeit zusammen und ordnet sie in den wissenschaftlichen Kontext ein. ?? rekapituliert die wesentlichen Erkenntnisse aus Konzeption, Implementierung und Evaluation. ?? beantwortet die zentrale Forschungsfrage auf Basis der gewonnenen Ergebnisse. ?? reflektiert kritisch die methodischen und technischen Einschränkungen der Arbeit. Abschließend gibt ?? einen Ausblick auf zukünftige Entwicklungsmöglichkeiten und offene Forschungsfragen.

5.1 Zusammenfassung der wesentlichen Erkenntnisse

Die vorliegende Arbeit untersuchte die Entwicklung und Performance-Optimierung eines Virtual-Tabletop-Plugins für Obsidian.md. Im Verlauf der Konzeption, Implementierung und Evaluation ergaben sich mehrere zentrale Erkenntnisse. Diese sind sowohl für das konkrete Projekt als auch für die allgemeine Plugin-Entwicklung in Electron-basierten Umgebungen relevant.

Architekturentscheidungen Die Wahl von PIXI.js v8 als Rendering-Engine ermöglicht konstante 120 FPS bei 100 gleichzeitig gerenderten Token. Die automatischen Optimierungen der WebGL-basierten Bibliothek – insbesondere Sprite-Batching und Texture-Caching – tragen zu dieser Performance bei. Die Entscheidung, PIXI.js v8 als eigenständige Dependency zu bundlen statt die in Obsidian global verfügbare v7-Version zu nutzen, war aufgrund der in ?? dokumentierten API-Änderungen und modernen Features der Version 8 notwendig.

Die Single-Bundle-Architektur von Obsidian-Plugins stellt eine strukturelle Einschränkung dar, die Code-Splitting und Lazy Loading erschwert. Alle Features müssen beim Plugin-Start geladen werden, was sich jedoch in den Messungen nicht als Engpass für die Runtime-Performance erwies. Der Haupteinfluss liegt im initialen Bundle-Parsing und Startup-Overhead.

Performance-Charakteristika Die automatisierten Benchmark-Messungen mit 500 Iterationen pro Szenario lieferten statistisch valide Daten zur Performance des Plugins:

- Die Rendering-Performance ist über alle getesteten Szenarien (0, 20, 100 Token) konstant und erreicht die Hardware-Obergrenze von 120 FPS
- Die Frame Time von durchschnittlich 8,33 ms liegt unter dem Schwellenwert für flüssige Interaktion (16,67 ms für 60 FPS [[Janzen2014FrameRate](#)])

- Der Speicherverbrauch skaliert mit der Token-Anzahl, wobei Texture-Sharing die Effizienz bei vielen gleichartigen Token erhöht

Identifizierte Probleme Die Langzeit-Benchmarks zeigten ein Memory-Leak bei intensiver Token-Manipulation. Mit einer Leak-Rate von 1,8 MB pro Iteration akkumuliert sich der Speicherverbrauch über längere Nutzungszeiträume. Ohne automatisierte Benchmarks mit hoher Iterationszahl wäre dieses Problem unentdeckt geblieben. Dies verdeutlicht die Notwendigkeit systematischer Performance-Evaluation.

5.2 Beantwortung der Forschungsfrage

Die zentrale Forschungsfrage dieser Arbeit lautete: Wie können Virtual-Tabletop-Plugins in der Electron-basierten Umgebung von Obsidian so implementiert werden, dass sie trotz der technischen Limitierungen eine für Echtzeit-Interaktionen ausreichende Performance erreichen?

Technische Machbarkeit Die Implementierung von Atlas VTT zeigt, dass performante VTT-Plugins in Obsidian realisierbar sind. Die gemessenen 120 FPS und Frame Times von 8,33 ms erfüllen die in Abschnitt 3.1 definierten Anforderungen von mehr als 30 FPS und weniger als 60 ms Interaktionslatenz für Dragging-Tasks. Die Electron-Umgebung stellt somit keine prinzipielle Barriere für Echtzeit-Rendering-Anwendungen dar.

Einflussfaktoren Drei technische Faktoren ermöglichen die erreichte Performance. Die in ?? beschriebene Integration von PIXI.js v8 mit WebGL-Backend nutzt GPU-Beschleunigung. Das automatische Sprite-Batching [PxiPerf2024] sowie die in ?? dokumentierte TilingSprite-basierte Grid-Implementierung reduzieren die Anzahl der Draw-Calls auf ein Minimum.

Das in ?? erläuterte Zustandsmanagement mit Zustand ermöglicht durch die reaktive Store-Architektur gezielte UI-Updates ohne vollständiges Re-Rendering. Die Trennung von Rendering-State und Persistenz-State vermeidet unnötige Disk-I/O während der Laufzeit. Die in ?? beschriebene asynchrone Ressourcenladung stellt sicher, dass Texturen den Main Thread nicht blockieren. Das Texture-Caching verhindert redundante Ladevorgänge bei wiederholter Token-Verwendung.

Einschränkungen Neben den erreichten Performance-Werten zeigt die Evaluation auch Grenzen auf. Das in ?? identifizierte Memory Leak bei intensiver Token-Manipulation deutet auf Komplexität im Lifecycle-Management von PIXI.js-Objekten hin. Die korrekte Freigabe von Graphics-Objekten, Event-Listnern und Texture-Referenzen erfordert präzise Implementierung. Die Bundle-Größe des Plugins von ca. 2,8 MB resultiert primär aus PIXI.js

und React-Abhängigkeiten. Die Single-Bundle-Architektur von Obsidian verhindert eine Aufteilung in bedarfsgesteuert geladene Module. Der Speicherverbrauch von ca. 10 MB pro Token kann bei Szenarien mit mehr als 100 Token relevant werden.

Fazit Die Forschungsfrage lässt sich wie folgt beantworten: VTT-Plugins können in Obsidian performant implementiert werden, wenn geeignete Technologien und Best Practices angewendet werden. WebGL-basiertes Rendering, reaktives State Management, asynchrones Loading, Texture-Caching und inkrementelle Updates bilden die technische Grundlage für flüssige Echtzeit-Interaktionen. Die technischen Limitierungen von Electron erweisen sich für die getesteten Szenarien mit bis zu 100 Token nicht als Engpass. Die Herausforderungen liegen weniger in der Rendering-Performance als vielmehr im korrekten Memory Management und der Bundle-Größen-Optimierung.

5.3 Limitationen und kritische Reflexion

Die Aussagekraft der Ergebnisse unterliegt methodischen und technischen Einschränkungen, die im Folgenden zusammengefasst und kritisch reflektiert werden.

5.3.1 Zusammenfassung der Limitationen

Die Limitationen dieser Arbeit gliedern sich in methodische Einschränkungen der Evaluation sowie technische Constraints der Obsidian-Plattform. Da die vorangegangenen Kapitel beide Aspekte behandeln, erfolgt hier eine zusammenfassende Einordnung mit Verweisen auf die relevanten Abschnitte.

Die methodischen Limitationen der Performance-Evaluation sind in ?? dokumentiert. Das Single-Platform-Testing auf Apple Silicon, die synthetischen Benchmark-Szenarien sowie die Abhängigkeit von der gebündelten Electron-Version schränken die Generalisierbarkeit der quantitativen Ergebnisse auf andere Hardware- und Software-Konfigurationen ein.

Die Anforderungsanalyse identifizierte und adressierte die technischen Plattform-Limitationen. Die Single-Bundle-Architektur von Obsidian verhindert Code-Splitting und Lazy Loading, wie in ?? als nicht-funktionale Anforderung NF3.1 dokumentiert. Der PIXI.js-Versionskonflikt zwischen der globalen v7-Installation und der benötigten v8-Version wird in ?? behandelt; die gewählte Lösung erhöht die Bundle-Größe um ca. 500 KB.

Eine weitere Einschränkung betrifft den Fokus auf Runtime-Performance. Die Evaluation erfasste primär Frame Rate und Speicherverbrauch während der Laufzeit. Andere Metriken wie Startup-Zeit, Bundle-Parse-Duration und Time-to-Interactive blieben unberücksichtigt, obwohl diese die wahrgenommene Nutzererfahrung beim Plugin-Start beeinflussen.

5.3.2 Kritische Würdigung

Beiträge der Arbeit Die Arbeit dokumentiert die Entwicklung eines VTT-Plugins in der Obsidian-Umgebung. Die Architekturentscheidungen – PIXI.js v8, Zustand und die modulare Service-Architektur – sind für ähnliche Plugin-Projekte nachvollziehbar. Die Identifikation des Memory Leaks verdeutlicht die Notwendigkeit von Langzeit-Tests bei der Plugin-Entwicklung, da dieser Fehler bei manuellen Kurztests nicht erkennbar gewesen wäre.

Schwächen und Verbesserungspotenzial Rückblickend hätten einige Aspekte anders angegangen werden können. Die Memory-Leak-Analyse hätte früher im Entwicklungsprozess erfolgen können, um den Fehler zeitnah zu beheben statt ihn lediglich zu dokumentieren. Ein Vergleich mit alternativen Rendering-Ansätzen wie Canvas-2D oder SVG hätte die Wahl von PIXI.js empirisch untermauern können. Zudem hätte die Evaluation von Cross-Platform-Tests auf Windows und Linux profitiert, um die Generalisierbarkeit der Ergebnisse zu erhöhen.

Gewonnene Erkenntnisse Die Entwicklung und Evaluation des Plugins lieferte mehrere übertragbare Erkenntnisse. Performance-Tests sollten von Beginn an in den Entwicklungsprozess integriert werden, nicht erst als finale Evaluation. Erst bei ausreichend vielen Wiederholungen – mehr als 100 Iterationen – werden nicht offensichtliche Probleme wie Memory Leaks sichtbar. Die strikte Trennung von Rendering-State und Persistenz-State vereinfacht sowohl Performance-Optimierung als auch Debugging, da Änderungen in einem Bereich den anderen nicht beeinflussen.

5.4 Ausblick

Aufbauend auf den Erkenntnissen dieser Arbeit ergeben sich mehrere Ansatzpunkte für zukünftige Arbeiten.

Offene Forschungsfragen Die in ?? diskutierte Evaluation auf Apple Silicon schränkt die Generalisierbarkeit der Ergebnisse ein. Für zukünftige Arbeiten ergibt sich die Frage, wie sich die Performance-Charakteristika auf anderen Hardware-Plattformen (Windows/Linux, Intel/AMD) verhalten. Ebenso bleibt zu untersuchen, wie die Performance bei Szenarien mit mehr als 100 Token und bei dauerhaften Spielsitzungen über mehrere Stunden skaliert.

Das in ?? identifizierte Memory Leak erfordert eine systematische Analyse des Lifecycle-Managements von PIXI.js-Objekten. Die Ergebnisse legen nahe, dass insbesondere die Freigabe von Graphics-Objekten und Event-Listnern bei Token-Manipulation überprüft werden muss.

Weiterentwicklung Eine mögliche Weiterentwicklung besteht in der Auslagerung rechenintensiver Operationen in Web Workers [ElectronPerformanceGuide2024]. Die in ?? beschriebenen Grid-Berechnungen und die Asset-Indexierung sind Kandidaten für diese Optimierung, da sie den Main Thread nicht blockieren sollten.

Die in ?? dokumentierten Architekturmuster – PIXI.js-Integration (??), Zustand-basiertes State Management (??) und die modulare Service-Architektur (??) – sind auf andere Obsidian-Plugins mit Rendering-Anforderungen übertragbar.

A Anhang

A.1 Implementierungsdetails

A.1.1 Plugin Lifecycle Integration

Die vollständige Plugin-Initialisierung aus `main.ts` mit allen Lifecycle-Hooks:

```
// main.ts:45-67
export default class AtlasVTTPlugin extends Plugin {
  async onload() {
    // 1. Register Custom View Types
    this.registerView("atlas-vtt", (leaf) => new AtlasView(leaf, this));

    // 2. Initialize Singleton Services
    this.assetService = AssetService.getInstance(this.app);
    this.linkService = new TokenStatblockLinkService(this.app);

    // 3. Register Commands & Ribbon Icons
    this.addRibbonIcon("map", "Open Atlas VTT", () => {
      this.activateView();
    });

    // 4. Register Event Handlers
    this.registerEvent(
      this.app.workspace.on("atlas-vtt:refresh-assets", () => {
        this.assetService.refreshMetadata();
      })
    );
  }

  async onunload() {
    // Cleanup: Detach views, close connections
    this.app.workspace.detachLeavesOfType("atlas-vtt");
  }
}
```

Listing A.1: Plugin Lifecycle Integration (vollständige Implementierung)

A.1.2 Map Data Format

Das vollständige `.atlasmap` JSON-Schema mit Beispieldaten:

```

{
  "version": 1,
  "name": "Dungeon Level 1",
  "background": {
    "image": "atlas-vtt/maps/dungeon1.jpg",
    "grid": {
      "type": "square",
      "size": 70,
      "distance": 5,
      "units": "ft"
    }
  },
  "objects": [
    {
      "type": "Token",
      "id": "uuid-abc-123",
      "x": 350,
      "y": 280,
      "image": "atlas-vtt/tokens/goblin.png",
      "rotation": 0,
      "size": 1,
      "statblockPath": "statblocks/Goblin.md"
    },
    {
      "type": "FogRegion",
      "id": "uuid-def-456",
      "shape": "rect",
      "x": 100,
      "y": 100,
      "width": 200,
      "height": 150
    }
  ]
}

```

Listing A.2: .atlasmap JSON Schema (vollständiges Beispiel)

A.1.3 MapObject Class Hierarchy

Die vollständige TypeScript-Klassenhierarchie für Map-Objekte aus `types.ts`:

```

// src/app/types.ts:89-125
abstract class MapObject {
  id: string;
  x: number;
  y: number;
  gmOnly?: boolean;
  draggable: boolean = true;
}

```

```

    abstract serialize(): object;
}

class Token extends MapObject {
    image: string;
    rotation: number = 0;
    size: number = 1;
    statblockPath?: string;

    serialize(): object {
        return { type: "Token", ...this };
    }
}

class FogRegion extends MapObject {
    shape: "rect" | "poly";
    points: number[];

    serialize(): object {
        return { type: "FogRegion", ...this };
    }
}

```

Listing A.3: MapObject Class Hierarchy (vollständige Implementierung)

A.1.4 AssetService Singleton

Die vollständige Singleton-Implementierung des AssetService aus AssetService.ts:

```

// src/app/services/AssetService.ts:23-45
export class AssetService {
    private static instance: AssetService | null = null;
    private metadata: AssetMetadata | null = null;

    private constructor(private app: App) {}

    public static getInstance(app: App): AssetService {
        if (!AssetService.instance) {
            AssetService.instance = new AssetService(app);
        }
        return AssetService.instance;
    }

    async getAssets(): Promise<Asset[]> {
        if (!this.metadata) {
            await this.loadMetadata();
        }
        return this.metadata.tokens.concat(

```

```

        this.metadata.maps,
        this.metadata.sounds
    );
}
}

```

Listing A.4: AssetService Singleton Implementation (vollständige Implementierung)

A.1.5 Bidirektionale Token-Statblock-Verlinkung

Der vollständige Quellcode der linkTokenToStatblock-Methode aus TokenStatblock-LinkService.ts:

```

// src/app/services/TokenStatblockLinkService.ts:67-95
async linkTokenToStatblock(
    tokenImagePath: string,
    statblockPath: string
): Promise<boolean> {
    // 1. Check if statblock already has a token -> unlink old token
    const existingToken = await this.getTokenLinkedToStatblock(statblockPath);
    if (existingToken) {
        await this.unlinkToken(existingToken);
    }

    // 2. Update AssetService metadata
    await this.assetService.updateAsset(tokenImagePath, {
        statblockPath: statblockPath
    });

    // 3. Update statblock frontmatter
    await this.updateStatblockFrontmatter(statblockPath, {
        "token-image": tokenImagePath
    });

    // 4. Emit events for synchronization
    this.emit("link-changed", { tokenImagePath, statblockPath });
    this.app.workspace.trigger("atlas-vtt:refresh-assets");

    return true;
}

```

Listing A.5: Bidirectional Linking Logic (vollständige Implementierung)

A.1.6 State-Management-Store

Die vollständige AtlasState-Interface-Definition und Store-Konfiguration aus atlasStore.ts:

```

// src/app/atlasStore.ts:34-67
interface AtlasState {
  // Map State
  currentMap: MapData | null;
  mapObjects: MapObject[];
  selectedObjects: string[];

  // Tool State
  activeTool: ToolType;

  // UI State
  sidebarOpen: boolean;
  assetBrowserTab: string;

  // Player Mode
  isPlayerMode: boolean;

  // Viewport State
  zoom: number;
  pan: { x: number; y: number };

  // Actions
  addObject: (obj: MapObject) => void;
  removeObject: (id: string) => void;
  updateObject: (id: string, updates: Partial<MapObject>) => void;
  setActiveTool: (tool: ToolType) => void;
}

export const useAtlasStore = create<AtlasState>()(
  temporal( // zundo middleware for undo/redo
    immer((set) => ({ // immer middleware for immutability
      currentMap: null,
      mapObjects: [],
      // ... initial state
      addObject: (obj) => set((state) => {
        state.mapObjects.push(obj);
      }),
    })),
  )
);

```

Listing A.6: Atlas Store Structure (vollständige Implementierung)

Ehrenwörtliche Erklärung

Hiermit versichere ich, dass die vorliegende Arbeit von mir selbstständig und ohne unerlaubte Hilfe angefertigt worden ist, insbesondere dass ich alle Stellen, die wörtlich oder annähernd wörtlich aus Veröffentlichungen entnommen sind, durch Zitate als solche gekennzeichnet habe. Ich versichere auch, dass die von mir eingereichte schriftliche Version mit der digitalen Version übereinstimmt. Weiterhin erkläre ich, dass die Arbeit in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde / Prüfungsstelle vorgelegen hat. Ich erkläre mich damit einverstanden/nicht einverstanden, dass die Arbeit der Öffentlichkeit zugänglich gemacht wird. Ich erkläre mich damit einverstanden, dass die Digitalversion dieser Arbeit zwecks Plagiatsprüfung auf die Server externer Anbieter hochgeladen werden darf. Die Plagiatsprüfung stellt keine Zurverfügungstellung für die Öffentlichkeit dar.

(Ort, Datum)

(Eigenhändige Unterschrift)